Subject : Machine learning

Name : Vasita Zeel

Roll no : 27

Supervised Learning

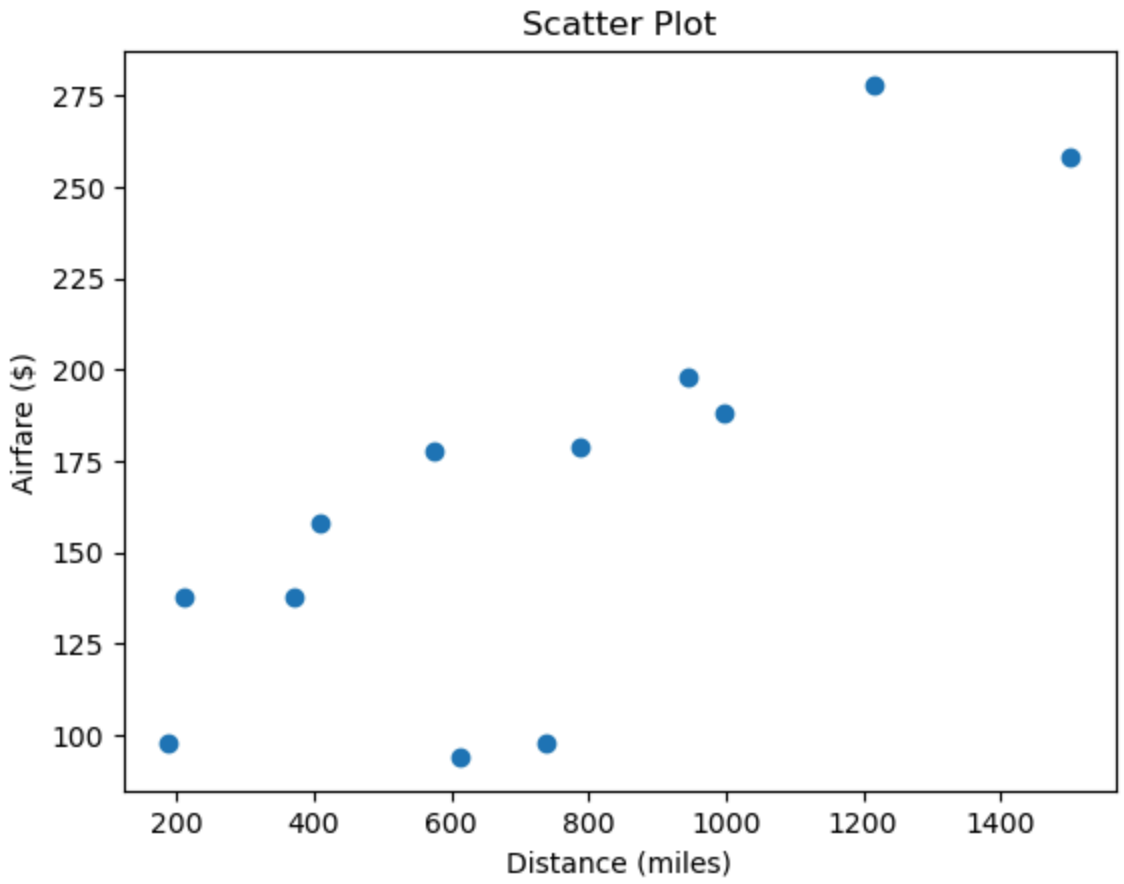
Linear Regression

Dataset - 1

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

In [3]:
```
# DATASET 1
# ------------------------------------------------
x = np.array([576,370,612,1216,409,1502,946,998,189,787,210,737])
y = np.array([178,138,94,278,158,258,198,188,98,179,138,98])
n = len(x)
```

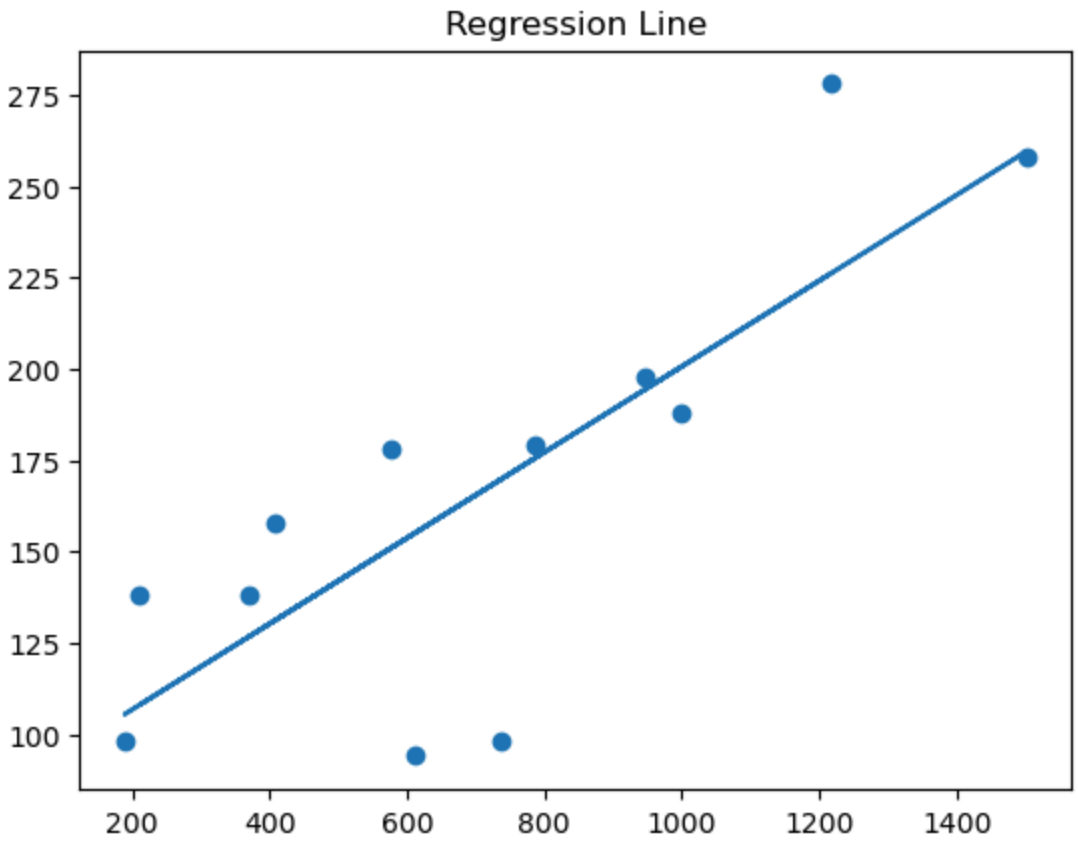
In [4]:
```
# (a) Scatter Plot
plt.scatter(x, y)
plt.xlabel("Distance (miles)")
plt.ylabel("Airfare ($)")
plt.title("Scatter Plot")
plt.show()
```



In [5]:
```
# (b) Regression using formula
sum_x = np.sum(x)
sum_y = np.sum(y)
sum_xy = np.sum(x*y)
sum_x2 = np.sum(x**2)
a1 = (n*sum_xy - sum_x*sum_y) / (n*sum_x2 - sum_x**2)
a0 = (sum_y - a1*sum_x) / n
print("Slope a1 =", a1)
print("Intercept a0 =", a0)
```

```
Slope a1 = 0.11737508839231386
Intercept a0 = 83.26735367241098
```

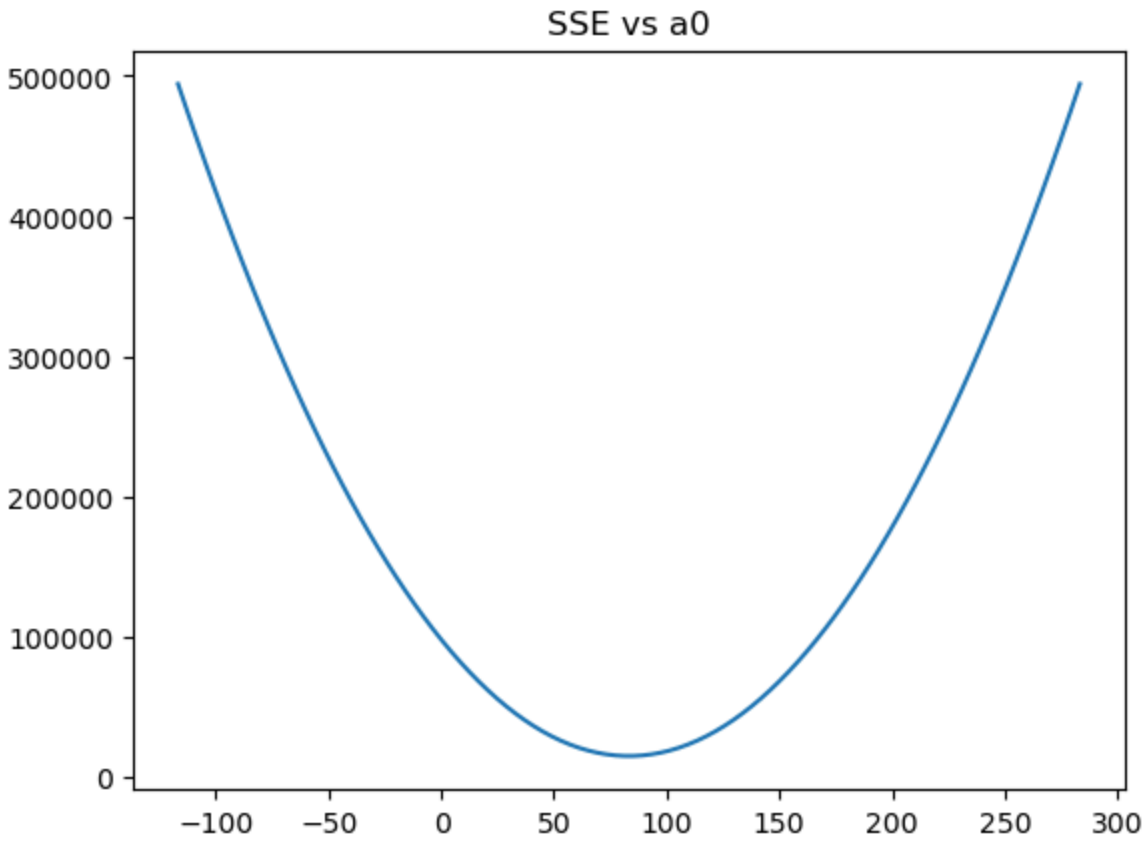
In [6]:
```
# Regression Line
y_pred = a0 + a1*x
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.title("Regression Line")
plt.show()
```



In [7]:
```
# (c) SSE
SSE = np.sum((y - y_pred)**2)
print("SSE =", SSE)
```

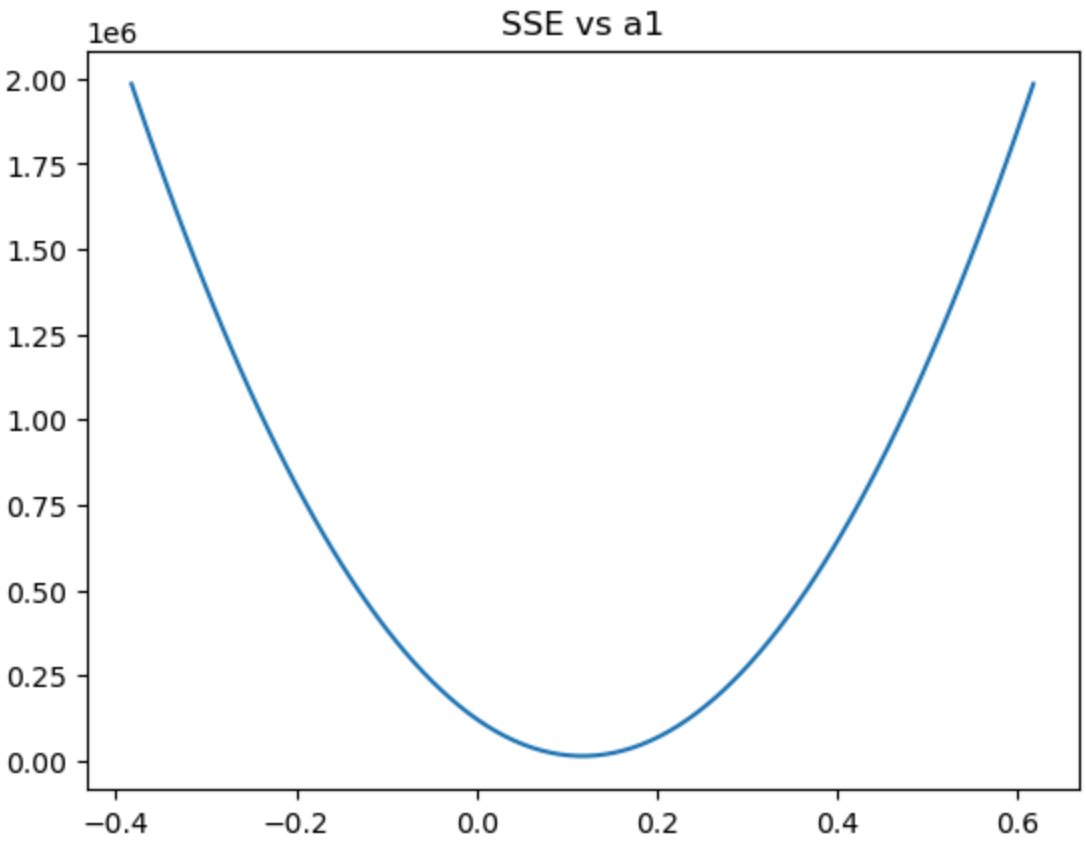
```
SSE = 14308.837785527729
```

In [8]:
```
# (d) S(a0)
a0_vals = np.linspace(a0-200, a0+200, 100)
sse_a0 = [np.sum((y - (v + a1*x))**2) for v in a0_vals]
plt.plot(a0_vals, sse_a0)
plt.title("SSE vs a0")
plt.show()
```

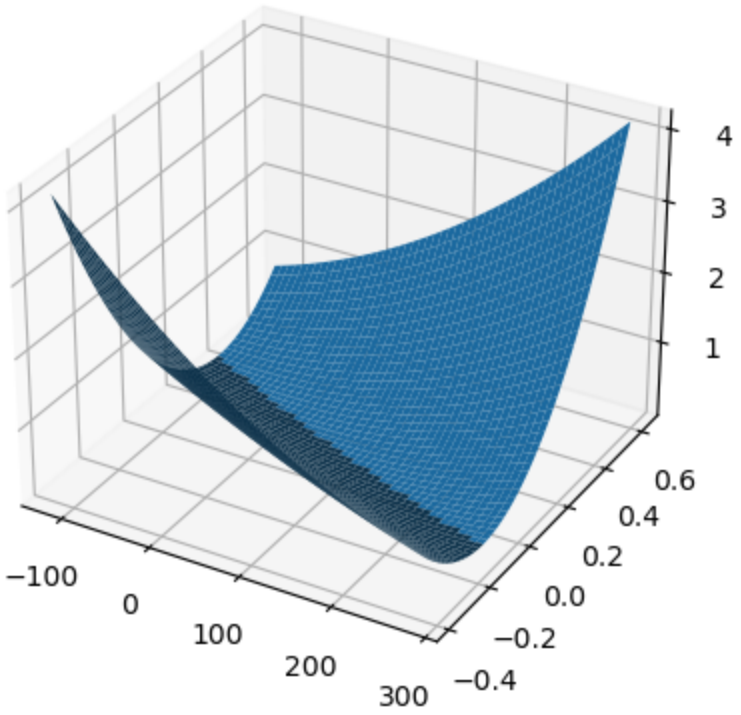


In [9]:
```
# (e) S(a1)
a1_vals = np.linspace(a1-0.5, a1+0.5, 100)
print(a1_vals)
sse_a1 = [np.sum((y - (a0 + v*x))**2) for v in a1_vals]
plt.plot(a1_vals, sse_a1)
plt.title("SSE vs a1")
plt.show()
```

```
[-0.38262491 -0.3725239  -0.36242289 -0.35232188 -0.34222087 -0.33211986
 -0.32201885 -0.31191784 -0.30181683 -0.29171582 -0.28161481 -0.2715138
 -0.26141279 -0.25131178 -0.24121077 -0.23110976 -0.22100875 -0.21090774
 -0.20080673 -0.19070572 -0.18060471 -0.1705037  -0.16040269 -0.15030168
 -0.14020067 -0.13009966 -0.11999865 -0.10989764 -0.09979663 -0.08969562
 -0.07959461 -0.0694936  -0.05939259 -0.04929158 -0.03919057 -0.02908956
 -0.01898855 -0.00888754  0.00121347  0.01131448  0.02141549  0.0315165
  0.04161751  0.05171852  0.06181953  0.07192054  0.08202155  0.09212256
  0.10222357  0.11232458  0.12242559  0.1325266   0.14262761  0.15272862
  0.16282963  0.17293064  0.18303165  0.19313266  0.20323367  0.21333468
  0.22343569  0.2335367   0.24363771  0.25373872  0.26383973  0.27394074
  0.28404176  0.29414277  0.30424378  0.31434479  0.3244458   0.33454681
  0.34464782  0.35474883  0.36484984  0.37495085  0.38505186  0.39515287
  0.40525388  0.41535489  0.4254559   0.43555691  0.44565792  0.45575893
  0.46585994  0.47596095  0.48606196  0.49616297  0.50626398  0.51636499
  0.526466    0.53656701  0.54666802  0.55676903  0.56687004  0.57697105
  0.58707206  0.59717307  0.60727408  0.61737509]
```



In [10]:
```
# (f) Surface Plot
A0, A1 = np.meshgrid(
np.linspace(a0-200, a0+200, 50),
np.linspace(a1-0.5, a1+0.5, 50)
)
S = np.zeros_like(A0)
for i in range(A0.shape[0]):
    for j in range(A0.shape[1]):
        S[i,j] = np.sum((y - (A0[i,j] + A1[i,j]*x))**2)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(A0, A1, S)
plt.show()
```



In [11]:
```
x_mean = np.mean(x)
x_std  = np.std(x)

x_scaled = (x - x_mean) / x_std

a0_gd, a1_gd = 0, 0
lr = 0.01

for _ in range(10000):
    y_gd = a0_gd + a1_gd * x_scaled

    da0 = -2 * np.sum(y - y_gd)
    da1 = -2 * np.sum(x_scaled * (y - y_gd))

    a0_gd -= lr * da0
    a1_gd -= lr * da1


# ---------------------------------------------------
# Convert back to original scale
# ---------------------------------------------------
a1_original = a1_gd / x_std
a0_original = a0_gd - a1_original * x_mean

print("GD (original scale):", a0_original, a1_original)
```

```
GD (original scale): 83.26735367241096 0.11737508839231385
```

In [12]:
```
# (h) Matrix Method
```

```
X = np.column_stack((np.ones(n), x))
theta = np.linalg.inv(X.T @ X) @ X.T @ y
print("Matrix:", theta)
```

```
Matrix: [83.26735367  0.11737509]
```

In [13]:
```
# (j) Metrics
SST = np.sum((y - np.mean(y))**2)
R2 = 1 - SSE/SST
MSE = SSE/n
MAE = np.mean(np.abs(y - y_pred))
RMSE = np.sqrt(MSE)
print("R2 =", R2)
print("MSE =", MSE)
print("MAE =", MAE)
print("RMSE =", RMSE)
```

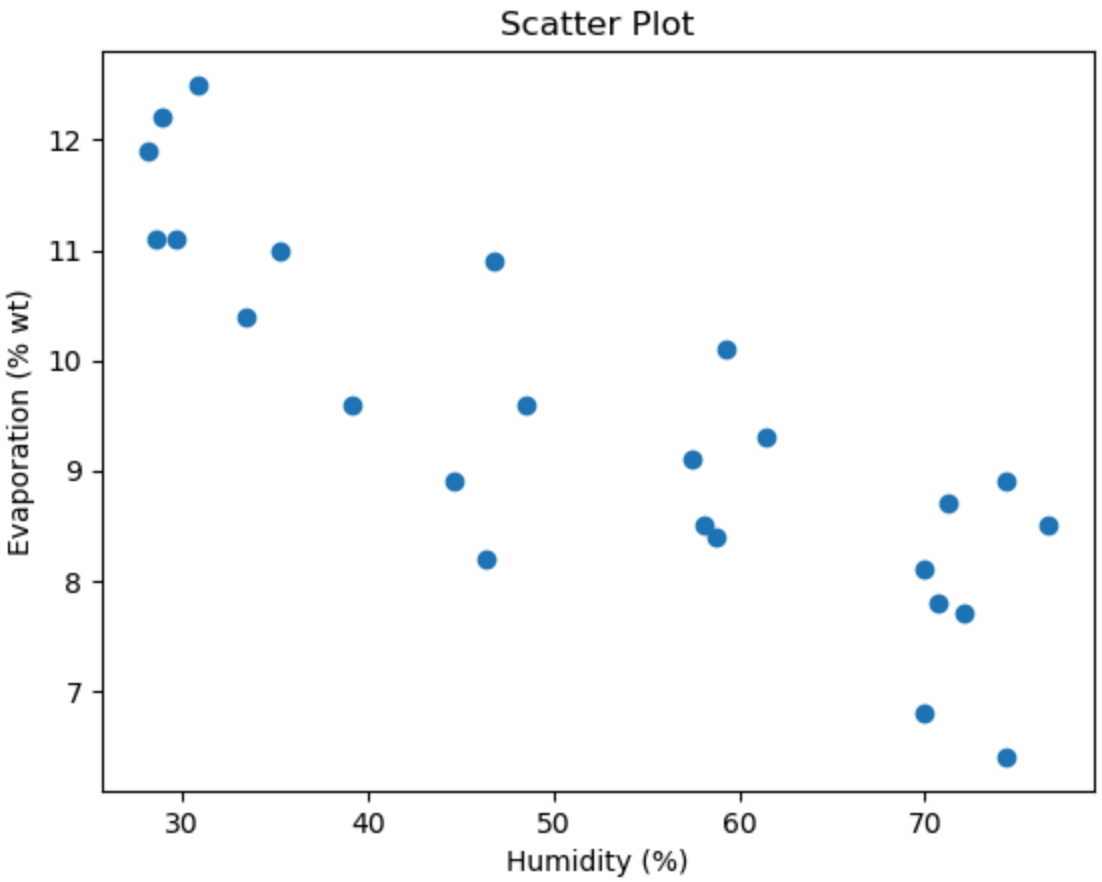
```
R2 = 0.6320019429562409
MSE = 1192.4031487939774
MAE = 25.716229215036382
RMSE = 34.53119095533743
```

Dataset - 2

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

In [2]:
```
# DATASET 2
# ----------------------------------------------------
x = np.array([35.3,29.7,30.8,58.8,61.4,71.3,74.4,76.7,70.7,57.5,
46.4,28.9,28.1,39.1,46.8,48.5,59.3,70.0,70.0,74.4,
72.1,58.1,44.6,33.4,28.6])

y = np.array([11.0,11.1,12.5,8.4,9.3,8.7,6.4,8.5,7.8,9.1,
8.2,12.2,11.9,9.6,10.9,9.6,10.1,8.1,6.8,8.9,
7.7,8.5,8.9,10.4,11.1])
n = len(x)
```

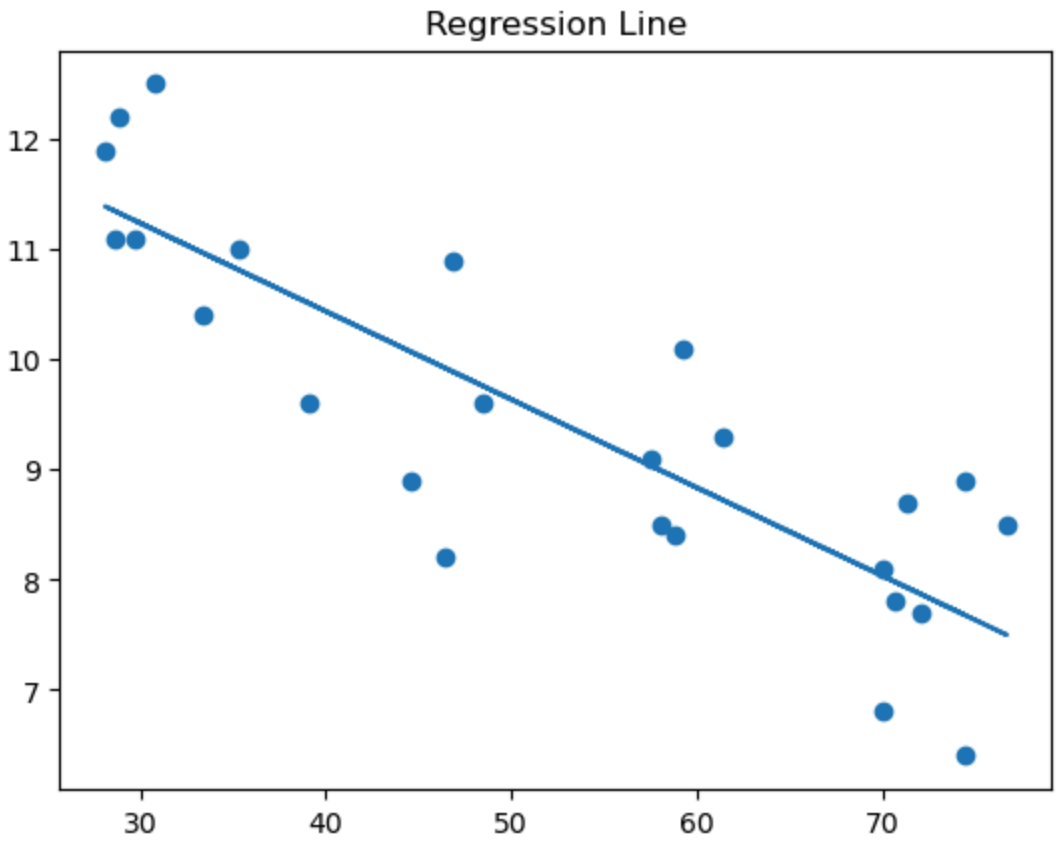
In [3]:
```
# (a) Scatter Plot
plt.scatter(x, y)
plt.xlabel("Humidity (%)")
plt.ylabel("Evaporation (% wt)")
plt.title("Scatter Plot")
plt.show()
```



In [4]:
```
# (b) Regression using formula
sum_x = np.sum(x)
sum_y = np.sum(y)
sum_xy = np.sum(x*y)
sum_x2 = np.sum(x**2)
a1 = (n*sum_xy - sum_x*sum_y) / (n*sum_x2 - sum_x**2)
a0 = (sum_y - a1*sum_x) / n
print("Slope a1 =", a1)
print("Intercept a0 =", a0)
```

```
Slope a1 = -0.08006059146778471
Intercept a0 = 13.638866868839605
```

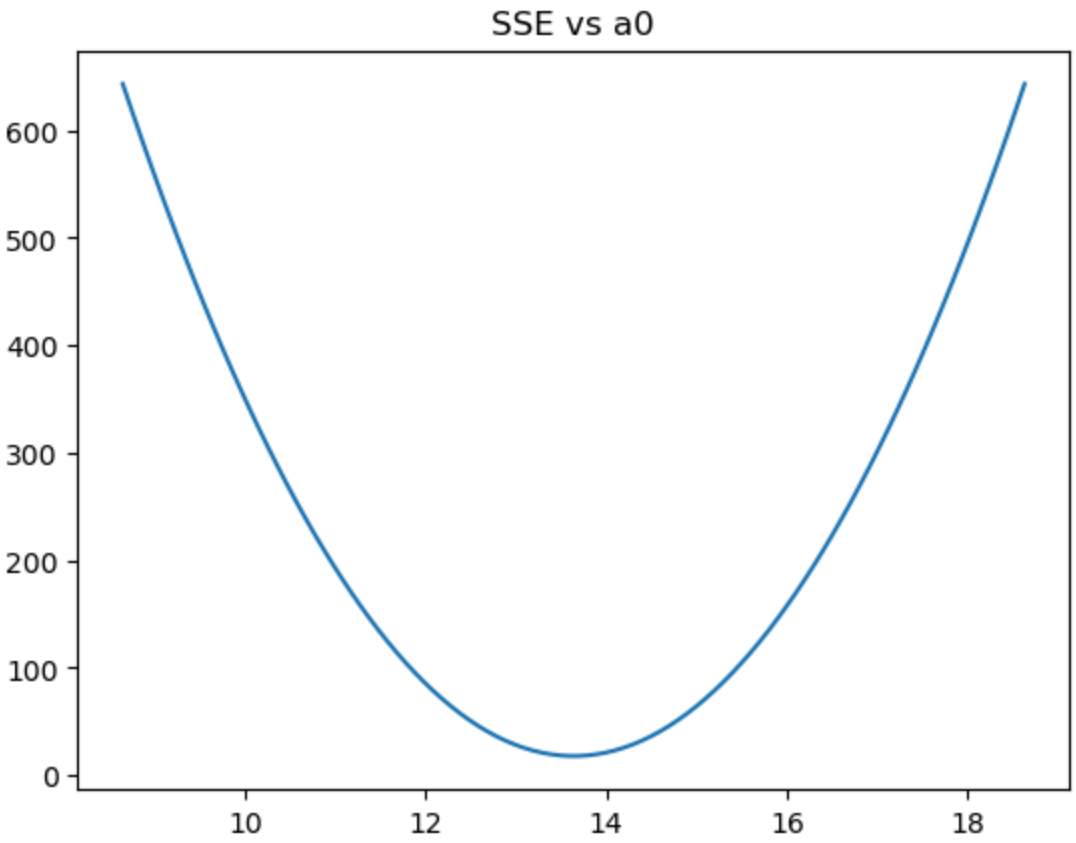
In [5]:
```
# Regression Line
y_pred = a0 + a1*x
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.title("Regression Line")
plt.show()
```



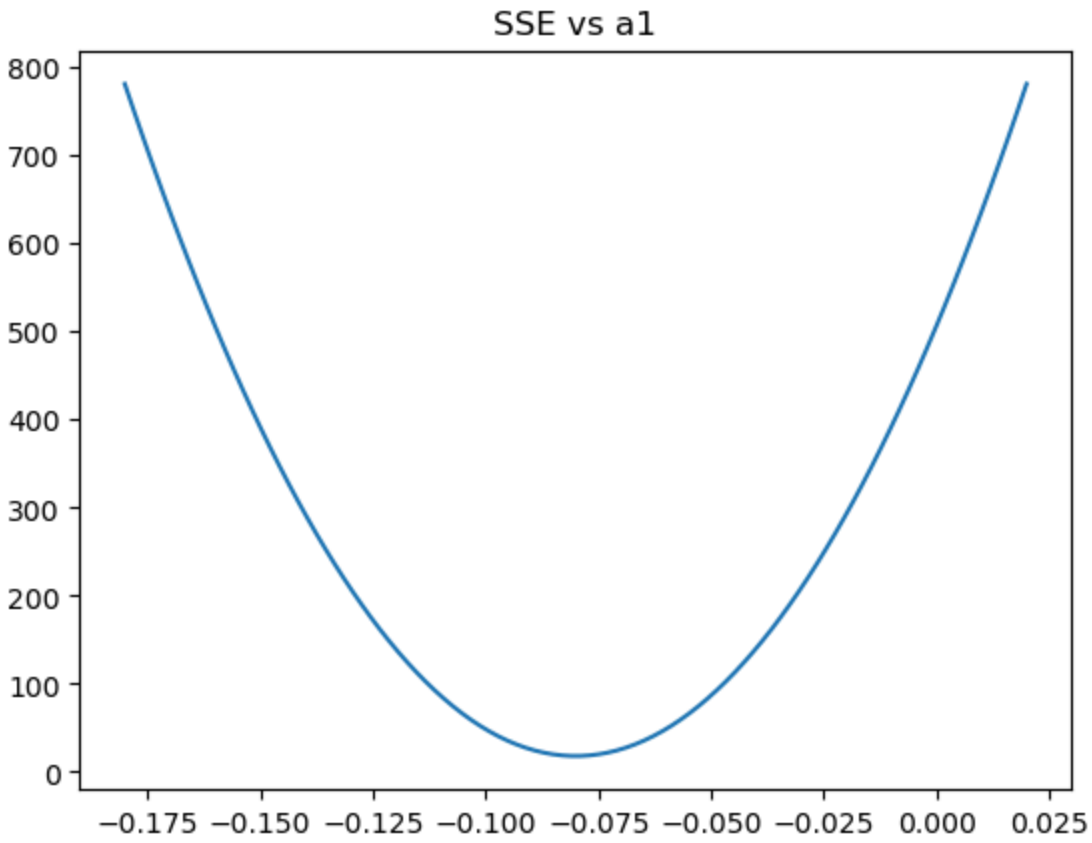
In [6]:
```
# (c) SSE
SSE = np.sum((y - y_pred)**2)
print("SSE =", SSE)
```

```
SSE = 18.060739189837232
```

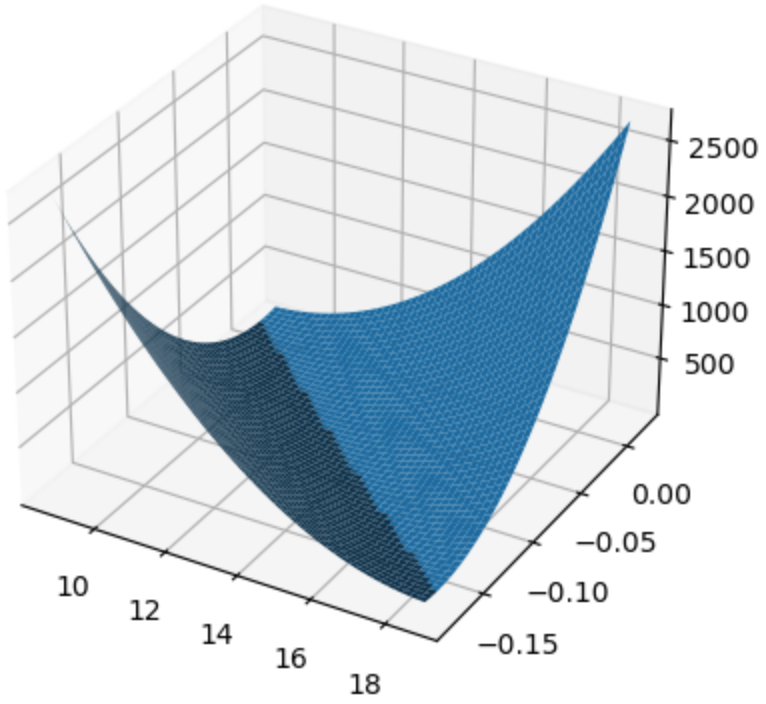
In [7]:
```
# (d) S(a0)
a0_vals = np.linspace(a0-5, a0+5, 100)
sse_a0 = [np.sum((y - (v + a1*x))**2) for v in a0_vals]
plt.plot(a0_vals, sse_a0)
plt.title("SSE vs a0")
plt.show()
```



In [8]:
```
# (e) S(a1)
a1_vals = np.linspace(a1-0.1, a1+0.1, 100)
sse_a1 = [np.sum((y - (a0 + v*x))**2) for v in a1_vals]
plt.plot(a1_vals, sse_a1)
plt.title("SSE vs a1")
plt.show()
```



In [9]:
```
# (f) Surface Plot
A0, A1 = np.meshgrid(
np.linspace(a0-5, a0+5, 50),
np.linspace(a1-0.1, a1+0.1, 50)
)
S = np.zeros_like(A0)
for i in range(A0.shape[0]):
    for j in range(A0.shape[1]):
        S[i,j] = np.sum((y - (A0[i,j] + A1[i,j]*x))**2)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(A0, A1, S)
plt.show()
```



In [10]:
```python
x_mean = np.mean(x)
x_std  = np.std(x)

x_scaled = (x - x_mean) / x_std

a0_gd, a1_gd = 0, 0
lr = 0.01

for _ in range(5000):
    y_gd = a0_gd + a1_gd * x_scaled

    da0 = -2 * np.sum(y - y_gd)
    da1 = -2 * np.sum(x_scaled * (y - y_gd))

    a0_gd -= lr * da0
    a1_gd -= lr * da1


# ---------------------------------------------------
# Convert back to original scale
# ---------------------------------------------------
a1_original = a1_gd / x_std
a0_original = a0_gd - a1_original * x_mean

print("GD (original scale):", a0_original, a1_original)
```

```
GD (original scale): 13.63886686883962 -0.08006059146778505
```

In [11]:
```python
# (h) Matrix Method
X = np.column_stack((np.ones(n), x))
```

```
theta = np.linalg.inv(X.T @ X) @ X.T @ y
print("Matrix:", theta)
```

```
Matrix: [13.63886687 -0.08006059]
```

In [12]:
```
# (j) Metrics
SST = np.sum((y - np.mean(y))**2)
R2 = 1 - SSE/SST
MSE = SSE/n
MAE = np.mean(np.abs(y - y_pred))
RMSE = np.sqrt(MSE)
print("R2 =", R2)
print("MSE =", MSE)
print("MAE =", MAE)
print("RMSE =", RMSE)
```

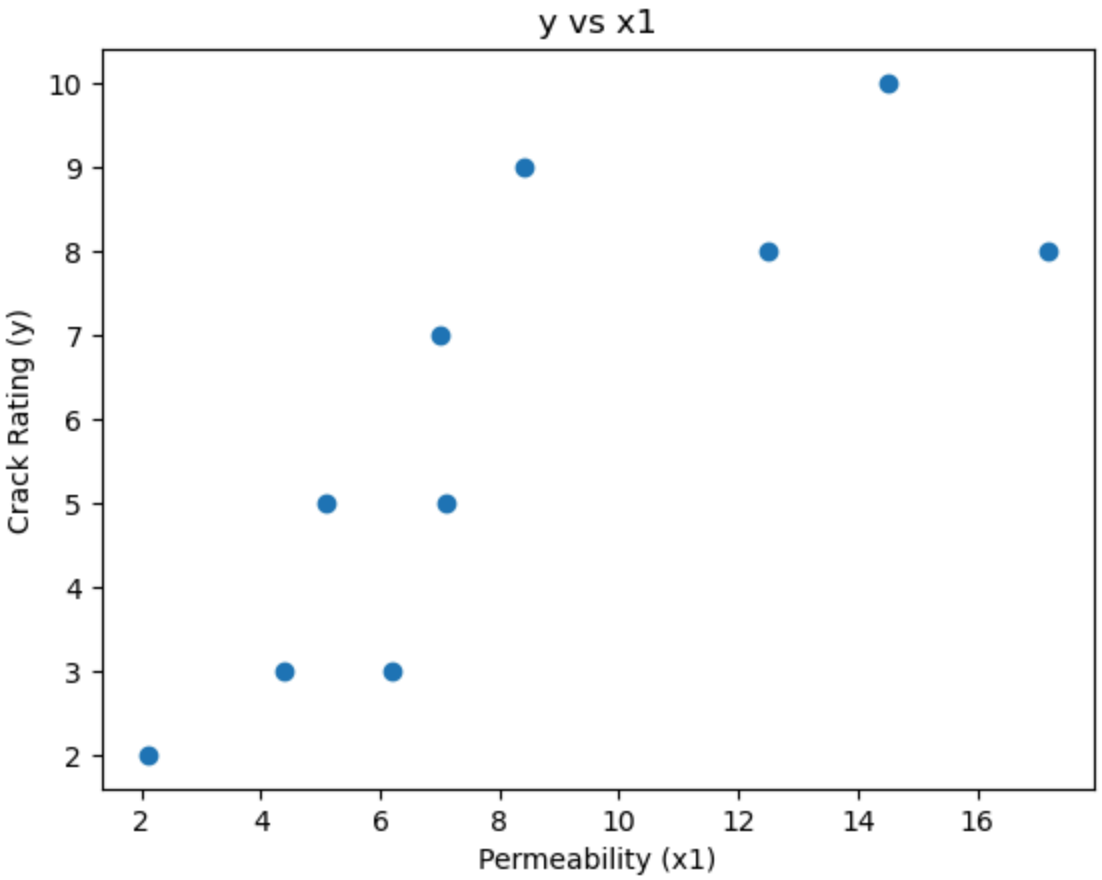
```
R2 = 0.717316855273449
MSE = 0.7224295675934893
MAE = 0.7049925127232681
RMSE = 0.8499585681628777
```

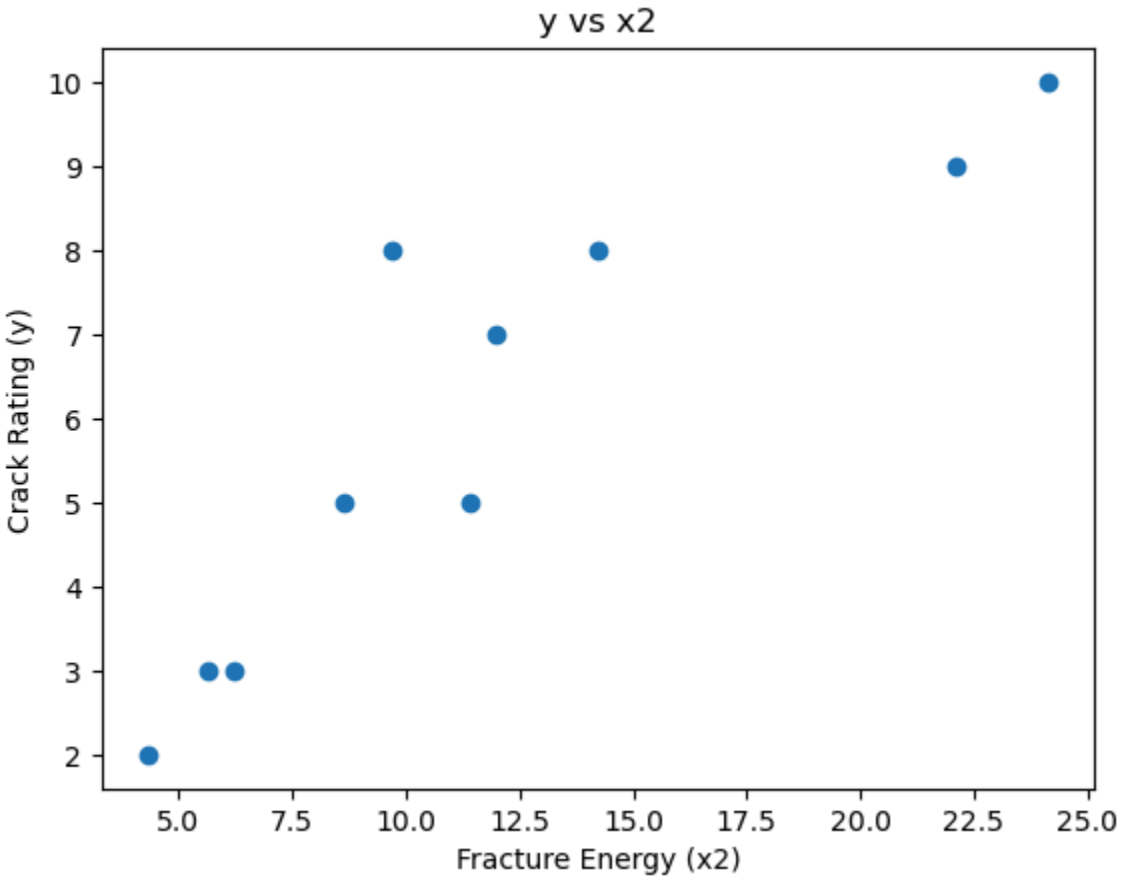
Dataset - 3

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

In [2]:
```
# DATASET
# ================================================
y  = np.array([2,9,5,10,3,3,8,7,8,5])
# Crack Rating
x1 = np.array([2.1,8.4,5.1,14.5,4.4,6.2,12.5,7.0,17.2,7.1])
 # Permeability
x2 = np.array([4.31,22.11,11.40,24.15,6.21,5.65,9.71,12.00,14.25,8.63])  # Fra
n = len(y)
```

In [3]:
```
# (a) Scatter Plots
# ================================================
plt.scatter(x1, y)
plt.xlabel("Permeability (x1)")
plt.ylabel("Crack Rating (y)")
plt.title("y vs x1")
plt.show()
plt.scatter(x2, y)
plt.xlabel("Fracture Energy (x2)")
plt.ylabel("Crack Rating (y)")
plt.title("y vs x2")
plt.show()
```

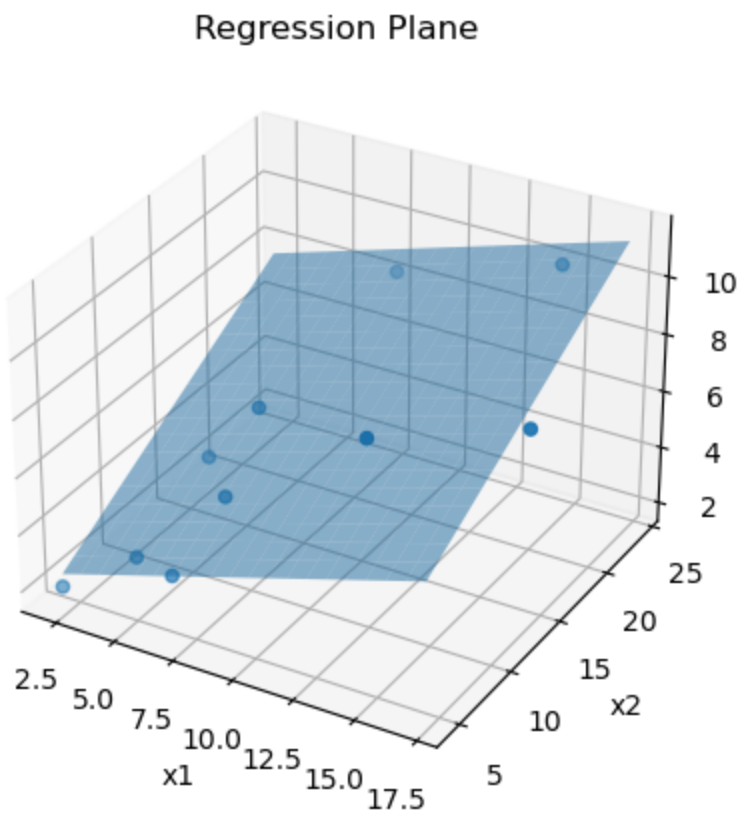

y vs x1
Crack Rating (y)
Permeability (x1)
10
9
8
7
6
5
4
3
2
2
4
6
8
10
12
14
16


y vs x2
Crack Rating (y)
Fracture Energy (x2)
10
9
8
7
6
5
4
3
2
5.0
7.5
10.0
12.5
15.0
17.5
20.0
22.5
25.0

In [4]:
```
# (b) Regression using formula (Matrix Form)
# ================================================
X = np.column_stack((np.ones(n), x1, x2))
theta = np.linalg.inv(X.T @ X) @ X.T @ y
print(theta)
a0, a1, a2 = theta
print("Slope a1 =", a1)
print("Slope a2 =", a2)
print("Intercept a0 =", a0)
print("Units:")
print("a1 -> change in y per unit x1")
print("a2 -> change in y per unit x2")
print("a0 -> y when x1=x2=0")
```

```
[0.8046745  0.24521109 0.26374699]
Slope a1 = 0.24521108935916924
Slope a2 = 0.26374698510779215
Intercept a0 = 0.8046744972685398
Units:
a1 -> change in y per unit x1
a2 -> change in y per unit x2
a0 -> y when x1=x2=0
```

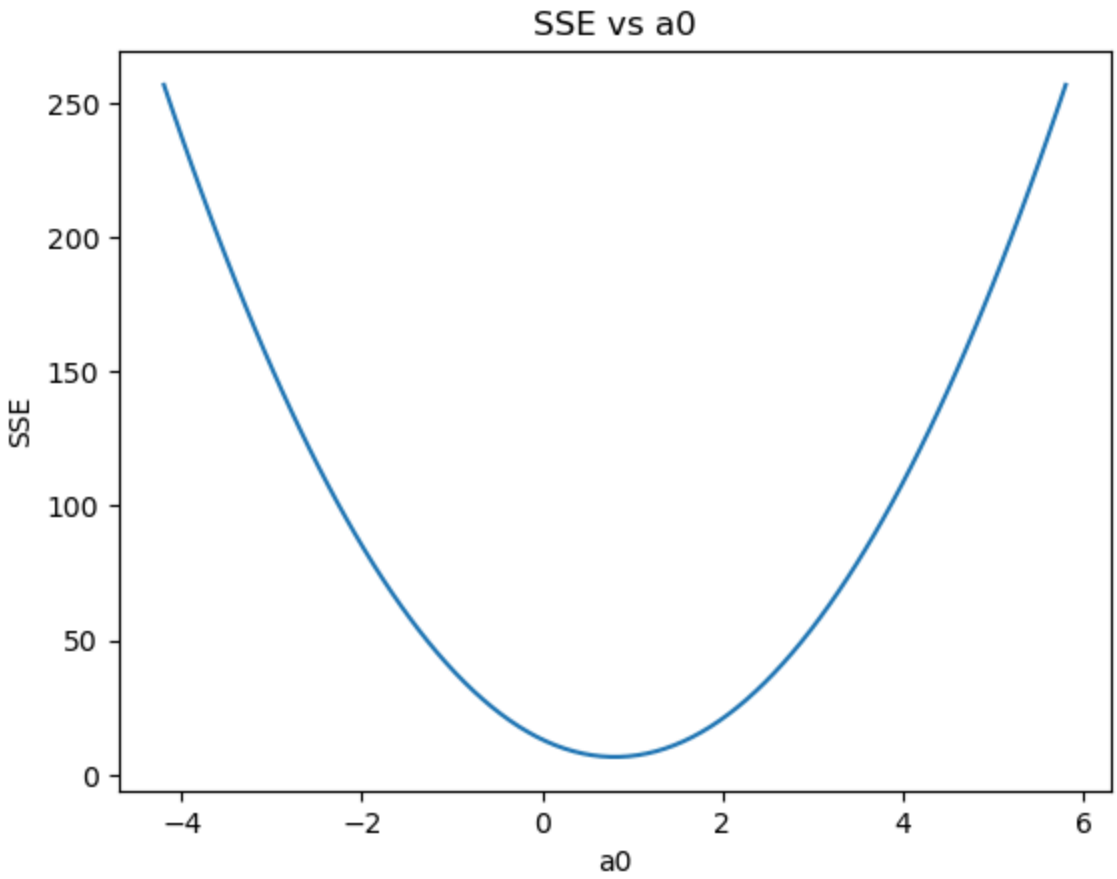
In [5]:
```
# (iii) Regression Plane
# ================================================
y_pred = X @ theta
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(x1, x2, y)
x1_grid, x2_grid = np.meshgrid(
np.linspace(min(x1), max(x1), 20),
np.linspace(min(x2), max(x2), 20)
)
y_plane = a0 + a1*x1_grid + a2*x2_grid
ax.plot_surface(x1_grid, x2_grid, y_plane, alpha=0.5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("y")
plt.title("Regression Plane")
plt.show()
```



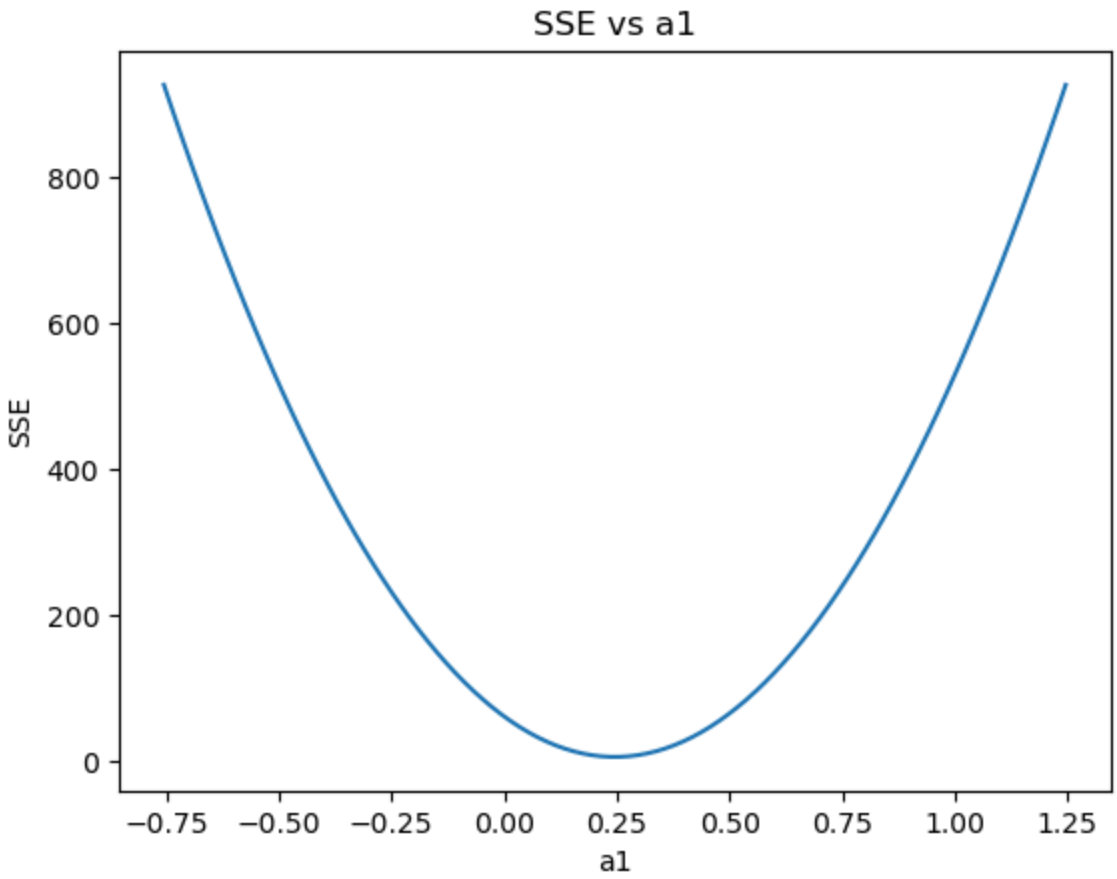
In [6]:
```
# (c) SSE
# ================================================
SSE = np.sum((y - y_pred)**2)
print("SSE =", SSE)
```

```
SSE = 6.603434348120017
```

In [7]:
```
# (d) S(a0)  (vary a0, fix a1,a2)
# ================================================
a0_vals = np.linspace(a0-5, a0+5, 100)
sse_a0 = []
for val in a0_vals:
    y_temp = val + a1*x1 + a2*x2
    sse_a0.append(np.sum((y - y_temp)**2))
plt.plot(a0_vals, sse_a0)
plt.xlabel("a0")
plt.ylabel("SSE")
plt.title("SSE vs a0")
plt.show()
```

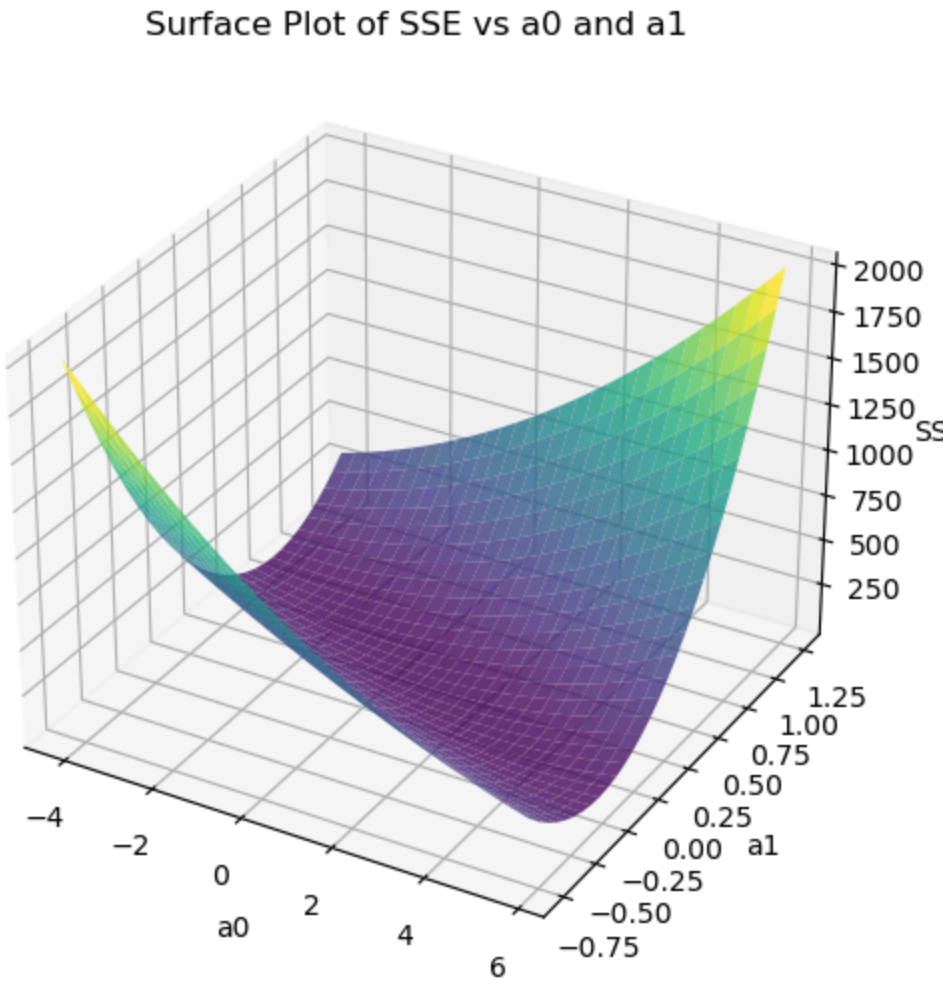


In [8]:
```
# (e) S(a1)  (vary a1, fix a0,a2)
# ================================================
a1_vals = np.linspace(a1-1, a1+1, 100)
sse_a1 = []
for val in a1_vals:
    y_temp = a0 + val*x1 + a2*x2
    sse_a1.append(np.sum((y - y_temp)**2))
plt.plot(a1_vals, sse_a1)
plt.xlabel("a1")
plt.ylabel("SSE")
plt.title("SSE vs a1")
plt.show()
```



In [9]:

```python
# (f) S(a0, a1) Surface (a2 fixed)
# ================================================
A0, A1 = np.meshgrid(
    np.linspace(a0 - 5, a0 + 5, 30),
    np.linspace(a1 - 1, a1 + 1, 30)
)

S = np.zeros_like(A0)

for i in range(A0.shape[0]):
    for j in range(A0.shape[1]):
        y_temp = A0[i, j] + A1[i, j] * x1 + a2 * x2
        S[i, j] = np.sum((y - y_temp) ** 2)

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(A0, A1, S, cmap='viridis', alpha=0.8)

ax.set_xlabel("a0")
ax.set_ylabel("a1")
ax.set_zlabel("SSE")
ax.set_title("Surface Plot of SSE vs a0 and a1")

plt.show()
```



In [10]:

```python
# FEATURE SCALING (Standardization)
# ------------------------------------------------
x1_mean, x1_std = np.mean(x1), np.std(x1)
x2_mean, x2_std = np.mean(x2), np.std(x2)

x1_s = (x1 - x1_mean) / x1_std
x2_s = (x2 - x2_mean) / x2_std


# ------------------------------------------------
# Gradient Descent on scaled data
# ------------------------------------------------
a0_gd, a1_gd, a2_gd = 0, 0, 0
lr = 0.01

for _ in range(10000):
    y_gd = a0_gd + a1_gd*x1_s + a2_gd*x2_s

    da0 = -2*np.sum(y - y_gd)
    da1 = -2*np.sum(x1_s*(y - y_gd))
    da2 = -2*np.sum(x2_s*(y - y_gd))

    a0_gd -= lr * da0
```

```
    a1_gd -= lr * da1
    a2_gd -= lr * da2


# ------------------------------------------------
# Convert coefficients back to original scale
# ------------------------------------------------
a1_orig = a1_gd / x1_std
a2_orig = a2_gd / x2_std

a0_orig = a0_gd - a1_orig*x1_mean - a2_orig*x2_mean


print("GD coefficients (original scale):")
print("a0 =", a0_orig)
print("a1 =", a1_orig)
print("a2 =", a2_orig)
```

```
GD coefficients (original scale):
a0 = 0.8046744972685516
a1 = 0.24521108935916852
a2 = 0.26374698510779204
```

In [11]:
```
# (h) Matrix Equation (already done)
# ================================================
print("Matrix Solution:", theta)
```

```
Matrix Solution: [0.8046745  0.24521109 0.26374699]
```

In [12]:
```
# (i) Verification
# ================================================
print("Formula/Matrix:", a0, a1, a2)
print("Gradient:", a0_gd, a1_gd, a2_gd)
```

```
Formula/Matrix: 0.8046744972685398 0.24521108935916924 0.26374698510779215
Gradient: 5.999999999999998 1.1143083693742215 1.677628126285202
```

In [13]:
```
# (j) Metrics
# ================================================
SST = np.sum((y - np.mean(y))**2)
R2 = 1 - SSE/SST
MSE = SSE/n
MAE = np.mean(np.abs(y - y_pred))
RMSE = np.sqrt(MSE)
print("\nMetrics:")
print("R2 =", R2)
print("MSE =", MSE)
print("MAE =", MAE)
print("RMSE =", RMSE)
```

```
Metrics:
R2 = 0.9056652235982855
MSE = 0.6603434348120018
MAE = 0.6730769028902384
RMSE = 0.8126151824892283
```

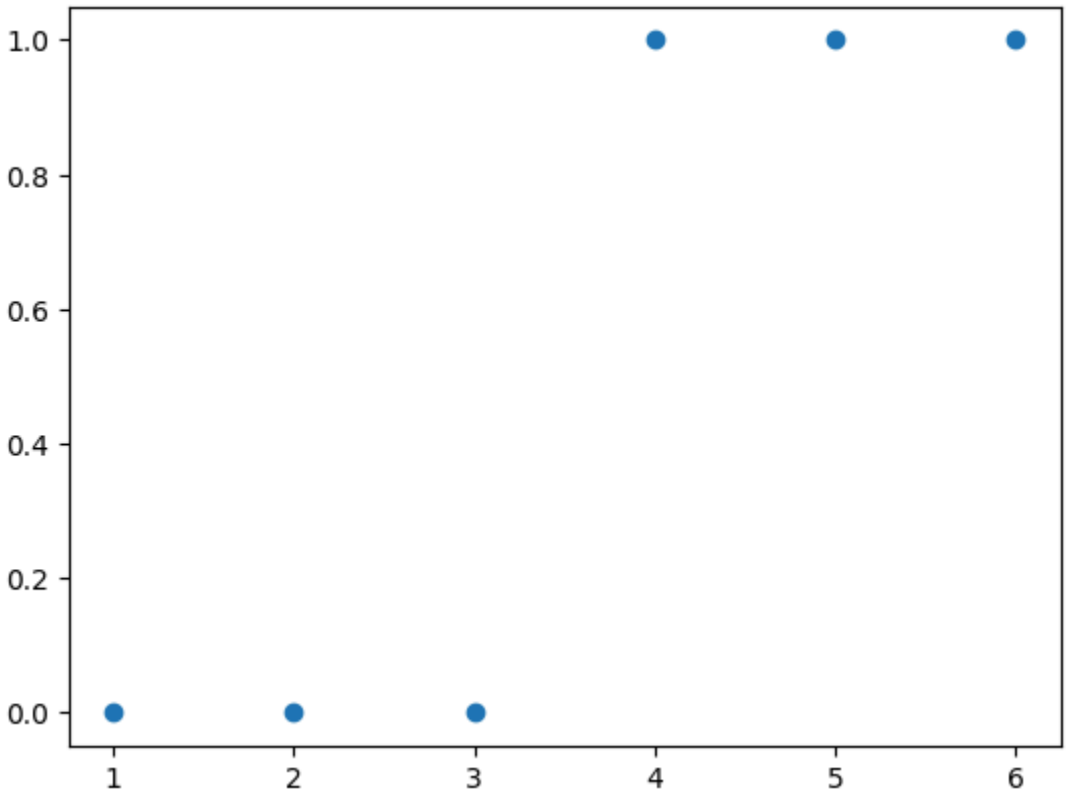
Logistic Regression

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [3]:
```
x = np.array([1,2,3,4,5,6])
y = np.array([0,0,0,1,1,1])

n = len(x)

alpha = 0.01
```

In [4]:
```
plt.scatter(x,y)
```

Out[4]:
```
<matplotlib.collections.PathCollection at 0x2c44b4cede0>
```



In [5]:
```
def sigmoid(x):
    return (1/(1+np.exp(-x)))
```

In [6]:
```
b0 = b1 = 0
```

In [11]:
```
epoch = 100
for i in range(epoch):
    z = b0 + b1*x
    p = sigmoid(z)
```

```
    loss=-(1/n)*(np.sum((y*np.log(p))+((1-y)*np.log(1-p))))
    print(loss)
    d_b0 = 1/n * (np.sum(p-y))

    d_b1 = 1/n * (np.sum((p-y)*x))

    b0 = b0 - alpha * d_b0
    b1 = b1 - alpha * d_b1


    print(f" epoch {i} \n b0 = {b0} b1 = {b1} \n\n")
```

0.5955104039219107
 epoch 0
 b0 = -0.121574915490487 b1 = 0.22557614870375495

0.5952320461189686
 epoch 1
 b0 = -0.12313365813101078 b1 = 0.22616781487859663

0.5949541960275668
 epoch 2
 b0 = -0.12469325732150191 b1 = 0.22675301120773173

0.594676835971276
 epoch 3
 b0 = -0.1262536635769687 b1 = 0.22733194287025255

0.5943999493928857
 epoch 4
 b0 = -0.1278148290622144 b1 = 0.2279048082025456

0.5941235207816925
 epoch 5
 b0 = -0.12937670753468067 b1 = 0.22847179893761638

0.5938475356056081
 epoch 6
 b0 = -0.13093925428941663 b1 = 0.2290331004353983

0.5935719802477588
 epoch 7
 b0 = -0.1325024261060852 b1 = 0.22958889190442325

0.5932968419472717
 epoch 8
 b0 = -0.1340661811979239 b1 = 0.23013934661521243

0.5930221087439631
 epoch 9
 b0 = -0.13563047916258072 b1 = 0.23068463210572987

0.5927477694266685
 epoch 10
 b0 = -0.13719528093474873 b1 = 0.23122491037922477

```
0.5924738134849639
 epoch 11
 b0 = -0.13876054874052754 b1 = 0.231760338094774


0.592200231064055
 epoch 12
 b0 = -0.1403262460534423 b1 = 0.23229106675082106


0.59192701292262
 epoch 13
 b0 = -0.14189233755205422 b1 = 0.232817242861995


0.5916541503934061
 epoch 14
 b0 = -0.14345878907909965 b1 = 0.23333900812947933


0.5913816353464003
 epoch 15
 b0 = -0.14502556760209762 b1 = 0.23385649960518898


0.5911094601543982
 epoch 16
 b0 = -0.14659264117536794 b1 = 0.23436984985000175


0.5908376176608153
 epoch 17
 b0 = -0.14815997890340524 b1 = 0.23487918708627953


0.590566101149587
 epoch 18
 b0 = -0.14972755090555592 b1 = 0.23538463534490423


0.5902949043170259
 epoch 19
 b0 = -0.15129532828194805 b1 = 0.23588631460704346


0.5900240212454984
 epoch 20
 b0 = -0.15286328308062577 b1 = 0.23638434094085162


0.5897534463788086
 epoch 21
 b0 = -0.15443138826584216 b1 = 0.23687882663330265
```

0.5894831744991706
 epoch 22
 b0 = -0.15599961768746645 b1 = 0.23736988031734296

0.5892132007056687
 epoch 23
 b0 = -0.15756794605146343 b1 = 0.2378576070945441

0.5889435203941056
 epoch 24
 b0 = -0.15913634889140427 b1 = 0.2383421086534277

0.588674129238149
 epoch 25
 b0 = -0.16070480254097022 b1 = 0.23882348338362752

0.5884050231716895
 epoch 26
 b0 = -0.16227328410741199 b1 = 0.23930182648604642

0.5881361983723349
 epoch 27
 b0 = -0.16384177144592899 b1 = 0.23977723007915977

0.5878676512459597
 epoch 28
 b0 = -0.16541024313493446 b1 = 0.2402497833016101

0.5875993784122489
 epoch 29
 b0 = -0.1669786784521737 b1 = 0.2407195724112318

0.5873313766911651
 epoch 30
 b0 = -0.16854705735166375 b1 = 0.24118668088063946

0.5870636430902829
 epoch 31
 b0 = -0.17011536044142472 b1 = 0.24165118948950703

0.5867961747929322
 epoch 32

```
 b0 = -0.17168356896197348 b1 = 0.24211317641366054


0.5865289691470984
 epoch 33
 b0 = -0.1732516647655522 b1 = 0.2425727173111018


0.5862620236550312
 epoch 34
 b0 = -0.1748196302960649 b1 = 0.24302988540507595


0.5859953359635149
 epoch 35
 b0 = -0.17638744856969638 b1 = 0.24348475156429056


0.5857289038547604
 epoch 36
 b0 = -0.17795510315618912 b1 = 0.24393738438039075


0.5854627252378728
 epoch 37
 b0 = -0.17952257816075431 b1 = 0.24438785024278956


0.5851967981408653
 epoch 38
 b0 = -0.1810898582065944 b1 = 0.24483621341094944


0.5849311207031784
 epoch 39
 b0 = -0.18265692841801526 b1 = 0.24528253608420691


0.5846656911686741
 epoch 40
 b0 = -0.18422377440410712 b1 = 0.24572687846922867


0.5844005078790739
 epoch 41
 b0 = -0.18579038224297398 b1 = 0.24616929884518382


0.5841355692678152
 epoch 42
 b0 = -0.1873567384664921 b1 = 0.24660985362671436


0.5838708738542921
```

```
 epoch 43
 b0 = -0.188922830045579 b1 = 0.24704859742478175


0.5836064202384652
 epoch 44
 b0 = -0.190488644375955 b1 = 0.24748558310546523


0.5833422070958099
 epoch 45
 b0 = -0.1920541692643799 b1 = 0.24792086184678458


0.5830782331725838
 epoch 46
 b0 = -0.19361939291534835 b1 = 0.24835448319361664


0.5828144972813942
 epoch 47
 b0 = -0.19518430391822794 b1 = 0.24878649511077292


0.5825509982970438
 epoch 48
 b0 = -0.1967488912348244 b1 = 0.24921694403430295


0.5822877351526401
 epoch 49
 b0 = -0.19831314418735926 b1 = 0.2496458749210852


0.5820247068359485
 epoch 50
 b0 = -0.1998770524468458 b1 = 0.2500733312967655


0.5817619123859763
 epoch 51
 b0 = -0.2014406060218492 b1 = 0.25049935530210043


0.5814993508897699
 epoch 52
 b0 = -0.20300379524761805 b1 = 0.25092398773776126


0.5812370214794165
 epoch 53
 b0 = -0.20456661077557414 b1 = 0.25134726810765123
```

0.5809749233292317
 epoch 54
 b0 = -0.2061290435631486 b1 = 0.25176923466078854

0.580713055653125
 epoch 55
 b0 = -0.20769108486395207 b1 = 0.2521899244318032

0.5804514177021303
 epoch 56
 b0 = -0.20925272621826804 b1 = 0.2526093732800965

0.5801900087620921
 epoch 57
 b0 = -0.21081395944385795 b1 = 0.2530276159277086

0.5799288281514969
 epoch 58
 b0 = -0.21237477662706758 b1 = 0.25344468599593817

0.5796678752194395
 epoch 59
 b0 = -0.21393517011422467 b1 = 0.2538606160407572

0.5794071493437183
 epoch 60
 b0 = -0.21549513250331762 b1 = 0.25427543758706206

0.5791466499290485
 epoch 61
 b0 = -0.21705465663594606 b1 = 0.25468918116179967

0.578886376405389
 epoch 62
 b0 = -0.21861373558953393 b1 = 0.25510187632600806

0.5786263282263722
 epoch 63
 b0 = -0.2201723626697963 b1 = 0.2555135517058072

0.5783665048678327
 epoch 64
 b0 = -0.22173053140345148 b1 = 0.2559242350223762

0.5781069058264301
 epoch 65
 b0 = -0.22328823553116994 b1 = 0.25633395312095075

0.5778475306183536
 epoch 66
 b0 = -0.22484546900075267 b1 = 0.256742731998874

0.5775883787781119
 epoch 67
 b0 = -0.22640222596053053 b1 = 0.25715059683273256

0.5773294498573959
 epoch 68
 b0 = -0.22795850075297783 b1 = 0.2575575720046087

0.5770707434240145
 epoch 69
 b0 = -0.22951428790853284 b1 = 0.25796368112747775

0.5768122590608948
 epoch 70
 b0 = -0.2310695821396182 b1 = 0.2583689470697801

0.576553996365146
 epoch 71
 b0 = -0.2326243783348547 b1 = 0.2587733919791946

0.576295954947182
 epoch 72
 b0 = -0.2341786715534621 b1 = 0.2591770373056407

0.5760381344298979
 epoch 73
 b0 = -0.2357324570198407 b1 = 0.25957990382353446

0.5757805344478975
 epoch 74
 b0 = -0.23728573011832757 b1 = 0.2599820116533237

0.5755231546467712
 epoch 75
 b0 = -0.23883848638812202 b1 = 0.2603833802823257

0.5752659946824155
 epoch 76
 b0 = -0.24039072151837418 b1 = 0.2607840285848914

0.5750090542203982
 epoch 77
 b0 = -0.241942431343432 b1 = 0.2611839748419175

0.5747523329353613
 epoch 78
 b0 = -0.2434936118382408 b1 = 0.2615832367597285

0.5744958305104613
 epoch 79
 b0 = -0.24504425911389097 b1 = 0.2619818314883497

0.5742395466368448
 epoch 80
 b0 = -0.24659436941330817 b1 = 0.26237977563919074

0.5739834810131564
 epoch 81
 b0 = -0.24814393910708238 b1 = 0.2627770853021595

0.5737276333450774
 epoch 82
 b0 = -0.24969296468943034 b1 = 0.26317377606222486

0.5734720033448932
 epoch 83
 b0 = -0.25124144277428745 b1 = 0.26356986301544666

0.5732165907310878
 epoch 84
 b0 = -0.252789370091525 b1 = 0.2639653607844901

0.5729613952279631
 epoch 85
 b0 = -0.25433674348328833 b1 = 0.26436028353364194

0.572706416565282
 epoch 86

```
 b0 = -0.255883559900452 b1 = 0.2647546449833439


0.5724516544779339
 epoch 87
 b0 = -0.2574298163991885 b1 = 0.26514845842426044


0.5721971087056215
 epoch 88
 b0 = -0.25897551013764664 b1 = 0.2655417367308948


0.5719427789925651
 epoch 89
 b0 = -0.2605206383727357 b1 = 0.26593449237476885


0.5716886650872282
 epoch 90
 b0 = -0.2620651984570125 b1 = 0.2663267374371807


0.5714347667420581
 epoch 91
 b0 = -0.26360918783566745 b1 = 0.266718483621554


0.5711810837132426
 epoch 92
 b0 = -0.2651526040436071 b1 = 0.2671097422653918


0.570927615760483
 epoch 93
 b0 = -0.2666954447026292 b1 = 0.2675005243518488


0.5706743626467803
 epoch 94
 b0 = -0.2682377075186881 b1 = 0.26789084052093287


0.5704213241382352
 epoch 95
 b0 = -0.2697793902792468 b1 = 0.26828070108034946


0.5701685000038608
 epoch 96
 b0 = -0.27132049085071364 b1 = 0.2686701160159995


0.5699158900154044
```

```
 epoch 97
 b0 = -0.2728610071759601 b1 = 0.2690590950021423


0.5696634939471855
 epoch 98
 b0 = -0.274400937271918 b1 = 0.2694476474112344


0.5694113115759385
 epoch 99
 b0 = -0.2759402792272523 b1 = 0.26983578232345456
```

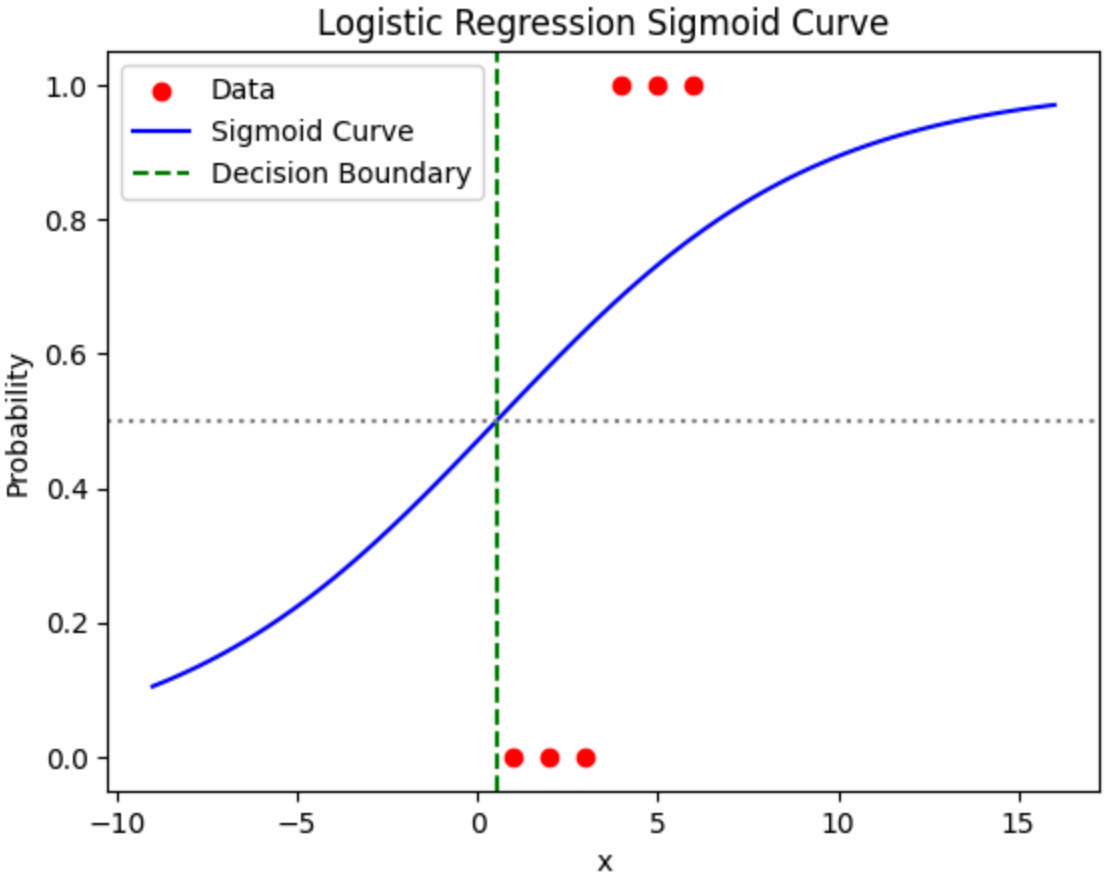
In [8]:
```
b0
```

Out[8]:
```
-0.1200170805940902
```

In [9]:
```
b1
```

Out[9]:
```
0.22497780041267243
```

In [10]:
```
x_vals = np.linspace(min(x)-10, max(x)+10, 100)
x_boundary = -b0 / b1

z_vals = b0 + b1 * x_vals
p_vals = sigmoid(z_vals)

plt.scatter(x, y, color='red', label='Data')

# Sigmoid curve
plt.plot(x_vals, p_vals, color='blue', label='Sigmoid Curve')


plt.axvline(x=x_boundary, color='green', linestyle='--', label='Decision Bound


plt.axhline(y=0.5, color='gray', linestyle=':')
plt.xlabel("x")
plt.ylabel("Probability")
plt.title("Logistic Regression Sigmoid Curve")
plt.legend()
plt.show()
```



k-Nearest Neighbour

Assignment 1

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# Data
X = np.array([[30,70],[22,90],[25,85],[28,65],[26,78]])
y = np.array(['Sunny','Rainy','Rainy','Sunny','Cloudy'])

query = np.array([24,80])

# Distance calculation
distances = []
for i in range(len(X)):
    dist = np.sqrt(np.sum((X[i] - query)**2))
    distances.append((dist, y[i]))

# Sort
distances.sort(key=lambda x: x[0])
```

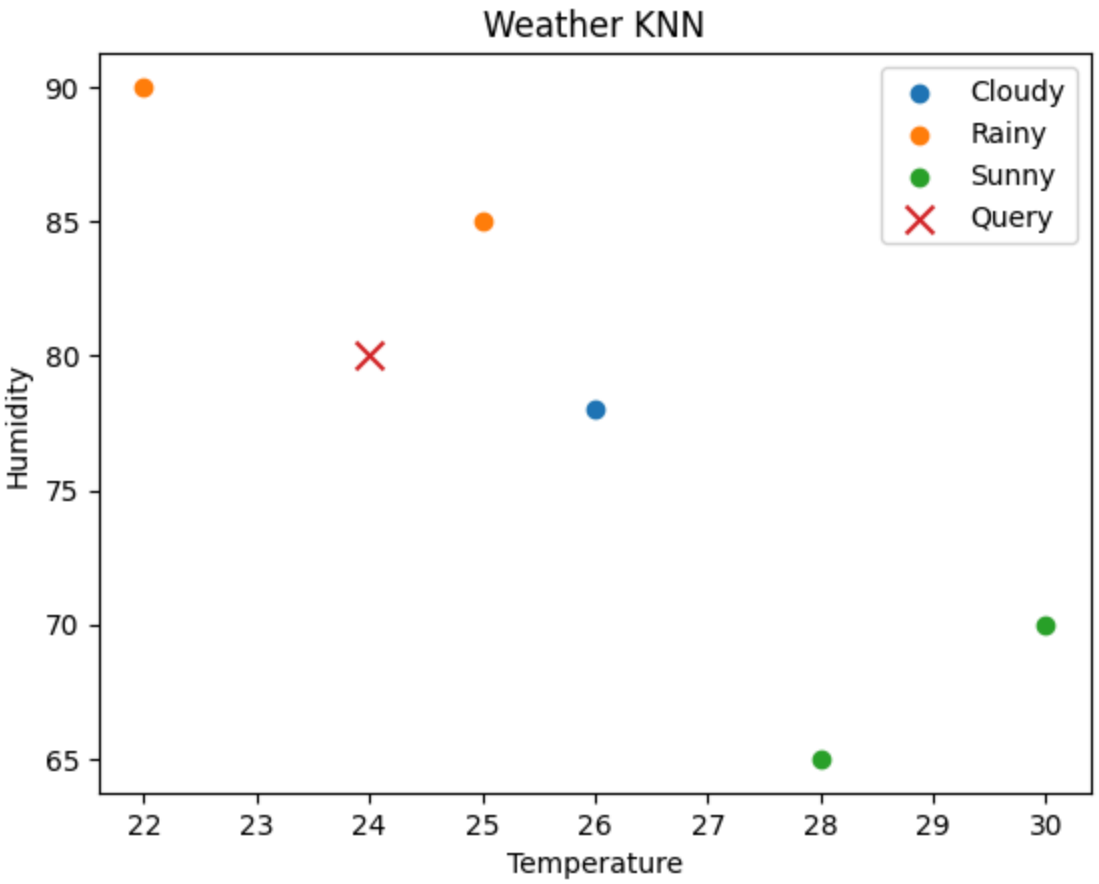
```
# K=3
k = 3
neighbors = distances[:k]

labels = [i[1] for i in neighbors]
prediction = Counter(labels).most_common(1)[0][0]

print("Scratch Prediction:", prediction)
```

```
Scratch Prediction: Rainy
```

In [2]:
```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
model.fit(X, y)

pred = model.predict([[24,80]])

print("Library Prediction:", pred[0])
```

```
Library Prediction: Rainy
```

In [3]:
```
plt.figure()

for label in np.unique(y):
    idx = y == label
    plt.scatter(X[idx,0], X[idx,1], label=label)

plt.scatter(24,80, marker='x', s=100, label='Query')

plt.xlabel("Temperature")
plt.ylabel("Humidity")
plt.title("Weather KNN")
plt.legend()

plt.show()
```



Assignment 2

In [1]:

```
import numpy as np
from collections import Counter

X = np.array([
    [7.0,2.5,0.2],
    [5.5,3.3,0.1],
    [6.2,2.9,0.3],
    [4.8,3.2,0.15]
])

y = np.array(['A','B','A','B'])

query = np.array([6.0,2.8,0.18])

# Min-Max Normalization
X_min = X.min(axis=0)
X_max = X.max(axis=0)

X_norm = (X - X_min) / (X_max - X_min)
query_norm = (query - X_min) / (X_max - X_min)

# Distance
distances = []
```

```
for i in range(len(X_norm)):
    dist = np.sqrt(np.sum((X_norm[i] - query_norm)**2))
    distances.append((dist, y[i]))

distances.sort(key=lambda x: x[0])

k = 3
labels = [i[1] for i in distances[:k]]
prediction = Counter(labels).most_common(1)[0][0]

print("Scratch Prediction:", prediction)
```

```
Scratch Prediction: A
```

In [2]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_scaled, y)

query_scaled = scaler.transform([query])
pred = model.predict(query_scaled)

print("Library Prediction:", pred[0])
```

```
Library Prediction: A
```

Assignment 3

In [1]:
```
import numpy as np
from collections import Counter
from sklearn import datasets

iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target

query = X[0]  # sample point

distances = []
for i in range(len(X)):
    dist = np.sqrt(np.sum((X[i] - query)**2))
    distances.append((dist, y[i]))

distances.sort(key=lambda x: x[0])

k = 3
labels = [i[1] for i in distances[:k]]
prediction = Counter(labels).most_common(1)[0][0]
```

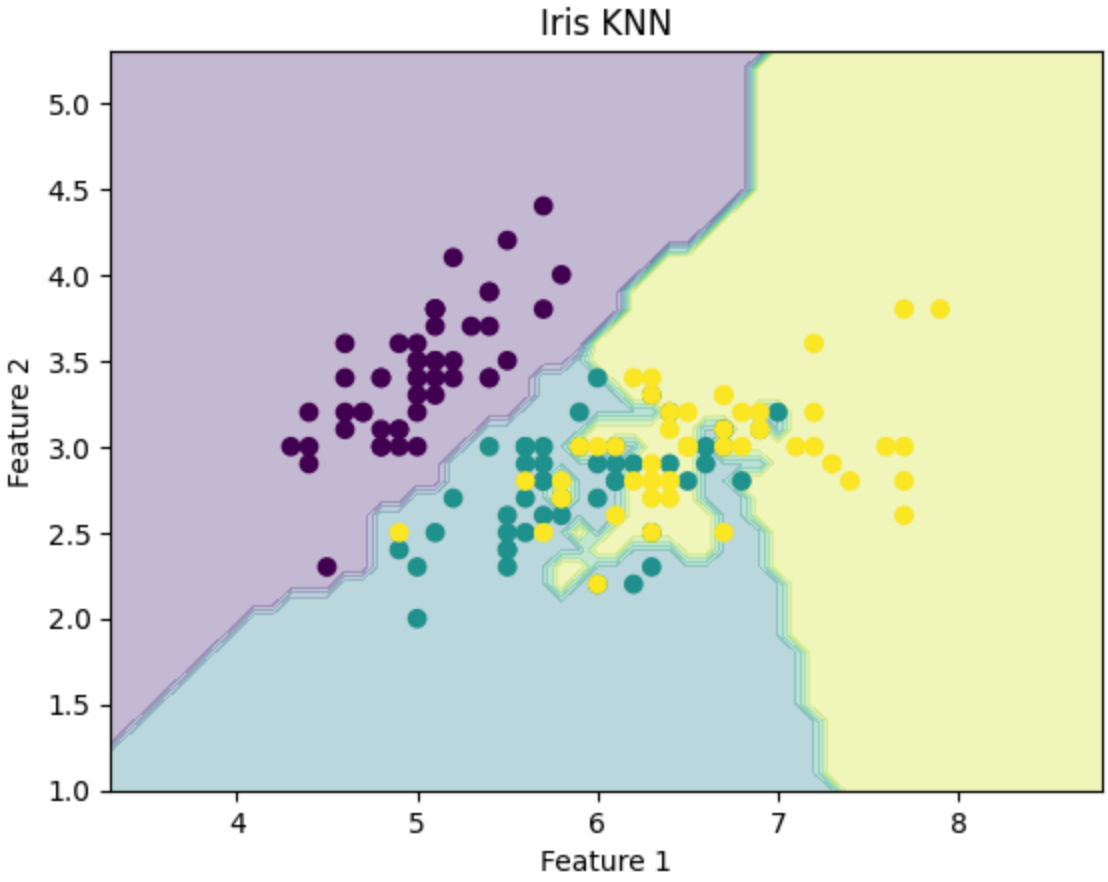
```
print("Scratch Prediction:", prediction)
```

```
Scratch Prediction: 0
```

In [2]:
```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

pred = model.predict([query])

print("Library Prediction:", pred[0])
```

```
Library Prediction: 0
```

In [3]:
```
import matplotlib.pyplot as plt

x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.1),
    np.arange(y_min, y_max, 0.1)
)

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure()
plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X[:,0], X[:,1], c=y)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Iris KNN")

plt.show()
```



Naive Bayes Model

Assignment 1

In [ ]:
```
import pandas as pd

# Create dataset
data = [
    ['yes','no','no','yes','mammals'],
    ['no','no','no','no','non-mammals'],
    ['no','no','yes','no','non-mammals'],
    ['yes','no','yes','no','mammals'],
    ['no','no','sometimes','yes','non-mammals'],
    ['no','no','no','yes','non-mammals'],
    ['yes','yes','no','yes','mammals'],
    ['no','yes','no','yes','non-mammals'],
    ['yes','no','no','yes','mammals'],
    ['yes','no','yes','no','non-mammals'],
    ['no','no','sometimes','yes','non-mammals'],
    ['no','no','sometimes','yes','non-mammals'],
    ['yes','no','no','yes','mammals'],
    ['no','no','yes','no','non-mammals'],
    ['no','no','sometimes','yes','non-mammals'],
```

```
    ['no','no','no','yes','non-mammals'],
    ['no','no','no','yes','mammals'],
    ['no','yes','no','yes','non-mammals'],
    ['yes','no','yes','no','mammals'],
    ['no','yes','no','yes','non-mammals']
]

columns = ['birth','fly','water','legs','class']
df = pd.DataFrame(data, columns=columns)

# Input
query = ['yes','no','yes','no']

# Step 1: Calculate prior probabilities
total = len(df)
classes = df['class'].unique()

prob = {}

for c in classes:
    df_c = df[df['class'] == c]
    prob_c = len(df_c) / total

    for i, col in enumerate(columns[:-1]):
        count = len(df_c[df_c[col] == query[i]])
        prob_c *= (count + 1) / (len(df_c) + len(df[col].unique()))  # Laplace

    prob[c] = prob_c

print("Probabilities:", prob)
print("Prediction:", max(prob, key=prob.get))
```

```
Probabilities: {'mammals': 0.02117283950617284, 'non-mammals': 0.00529629629629
6296}
Prediction: mammals
```

In [ ]:

```
# using library
from sklearn.preprocessing import OrdinalEncoder
from sklearn.naive_bayes import CategoricalNB
import pandas as pd

df = pd.DataFrame(data, columns=columns)

X = df.iloc[:, :-1]
y = df['class']

# SAME encoder for train + test
enc = OrdinalEncoder()
X_encoded = enc.fit_transform(X)

# Model
model = CategoricalNB()
model.fit(X_encoded, y)
```

```
# Encode query using SAME encoder
query = pd.DataFrame([['yes','no','yes','no']], columns=columns[:-1])
query_encoded = enc.transform(query)

pred = model.predict(query_encoded)

print("Library Prediction:", pred[0])
```

```
Prediction: [1]
```

Assignment 2

In [1]:

```
import pandas as pd

data2 = [
    ['brown','ragged','small','well-behaved'],
    ['black','ragged','big','dangerous'],
    ['black','smooth','big','dangerous'],
    ['black','curly','small','well-behaved'],
    ['white','curly','small','well-behaved'],
    ['white','smooth','small','dangerous'],
    ['red','ragged','big','well-behaved']
]

cols2 = ['color','fur','size','class']
df2 = pd.DataFrame(data2, columns=cols2)

query = ['black','ragged','big']

total = len(df2)
classes = df2['class'].unique()

prob = {}

for c in classes:
    df_c = df2[df2['class'] == c]
    prob_c = len(df_c) / total

    for i, col in enumerate(cols2[:-1]):
        count = len(df_c[df_c[col] == query[i]])
        prob_c *= (count + 1) / (len(df_c) + len(df2[col].unique()))

    prob[c] = prob_c

print("Probabilities:", prob)
print("Prediction:", max(prob, key=prob.get))
```

```
Probabilities: {'well-behaved': 0.020408163265306117, 'dangerous': 0.0367346938
7755102}
Prediction: dangerous
```

In [2]:

```
# using library
from sklearn.preprocessing import OrdinalEncoder
```

```
from sklearn.naive_bayes import CategoricalNB
import pandas as pd

df2 = pd.DataFrame(data2, columns=cols2)

X = df2.iloc[:, :-1]
y = df2['class']

enc = OrdinalEncoder()
X_enc = enc.fit_transform(X)

model = CategoricalNB()
model.fit(X_enc, y)

query = pd.DataFrame([['black','ragged','big']], columns=cols2[:-1])
query_enc = enc.transform(query)

pred = model.predict(query_enc)

print("Library Prediction:", pred[0])
```

```
Library Prediction: dangerous
```

Support Vector Machine

Assignment 1

In [1]:

```
import numpy as np
from sklearn.svm import SVC

# Data
X = np.array([
    [2,2],
    [4,4],
    [4,0],
    [0,0]
])

y = np.array([1,1,-1,-1])

# Model (linear SVM)
model = SVC(kernel='linear')
model.fit(X, y)

# Parameters
w = model.coef_[0]
b = model.intercept_[0]

print("w:", w)
print("b:", b)

# Support vectors
```

```
print("Support Vectors:\n", model.support_vectors_)

# Margin
margin = 2 / np.linalg.norm(w)
print("Margin:", margin)

# Verify constraints
for i in range(len(X)):
    val = y[i] * (np.dot(w, X[i]) + b)
    print(f"Constraint {i+1}:", val)
```

```
w: [0. 1.]
b: -1.0
Support Vectors:
 [[4. 0.]
 [0. 0.]
 [2. 2.]]
Margin: 2.0
Constraint 1: 1.0
Constraint 2: 3.0
Constraint 3: 1.0
Constraint 4: 1.0
```

Assignment 2

In [1]:

```
import numpy as np
from sklearn.svm import SVC

X = np.array([
    [3,1],[3,-1],[6,1],[6,-1],
    [1,0],[0,1],[0,-1],[-1,0]
])

y = np.array([1,1,1,1,-1,-1,-1,-1])

model = SVC(kernel='linear')
model.fit(X, y)

w = model.coef_[0]
b = model.intercept_[0]

print("w:", w)
print("b:", b)

print("Support Vectors:\n", model.support_vectors_)

margin = 2 / np.linalg.norm(w)
print("Margin:", margin)

# Check constraints
for i in range(len(X)):
    val = y[i] * (np.dot(w, X[i]) + b)
    print(f"Constraint {i+1}:", val)
```

```
w: [ 1.00048166e+00 -1.66533454e-16]
b: -2.001123875432526
Support Vectors:
 [[ 1.  0.]
 [ 3.  1.]
 [ 3. -1.]]
Margin: 1.9990371419717565
Constraint 1: 1.0003211072664357
Constraint 2: 1.0003211072664357
Constraint 3: 4.001766089965397
Constraint 4: 4.001766089965397
Constraint 5: 1.000642214532872
Constraint 6: 2.001123875432526
Constraint 7: 2.001123875432526
Constraint 8: 3.0016055363321796
```

Assignment 3

In [1]:
```python
import numpy as np

# Define phi(x)
def phi(x):
    x1, x2 = x
    return np.array([
        1,
        2*x1,
        2*x2,
        x1**2,
        x2**2,
        2*x1*x2
    ])

# Given values
x = np.array([1,2])
y = np.array([3,4])

# LHS
lhs = np.dot(phi(x), phi(y))

# RHS
rhs = (1 + np.dot(x, y))**2

print("Phi(x).Phi(y):", lhs)
print("(1 + x.y)^2:", rhs)
```

```
Phi(x).Phi(y): 214
(1 + x.y)^2: 144
```

In [ ]:
```python

```

Unsupervised Learning

K-Means

Hardcoded

In [13]:
```
import numpy as np
import pandas as pd
import math
x1 = [1,2,11,8,5,-1,0,-4,-5,-10]
x2 = [2,5,7,3,2,3,2,6,-9,0]
df = pd.DataFrame()
# df = pd.DataFrame(index=[i for i in range (1,11)])
df["x1"] = x1
df["x2"] = x2
df
```

Out[13]:

	x1	x2
0	1	2
1	2	5
2	11	7
3	8	3
4	5	2
5	-1	3
6	0	2
7	-4	6
8	-5	-9
9	-10	0

In [14]:
```
import math
def eg(p1,p2):
    sum=0
    for i in range(len(p1)):
        sum = sum + (p1[i]-p2[i])**2
    return math.sqrt(sum)
p1 = [1,2]
p2 = [11,7]
eg(p1,p2)
```

Out[14]:
```
11.180339887498949
```

In [15]:
```
k = 2
```

```
ind = np.random.randint(0,9,size=2)
ind = [2,6]
```

In [16]:
```
c1_cent = [x1[ind[0]],x2[ind[0]]]
c2_cent = [x1[ind[1]],x2[ind[1]]]
print(c1_cent,c2_cent)
```

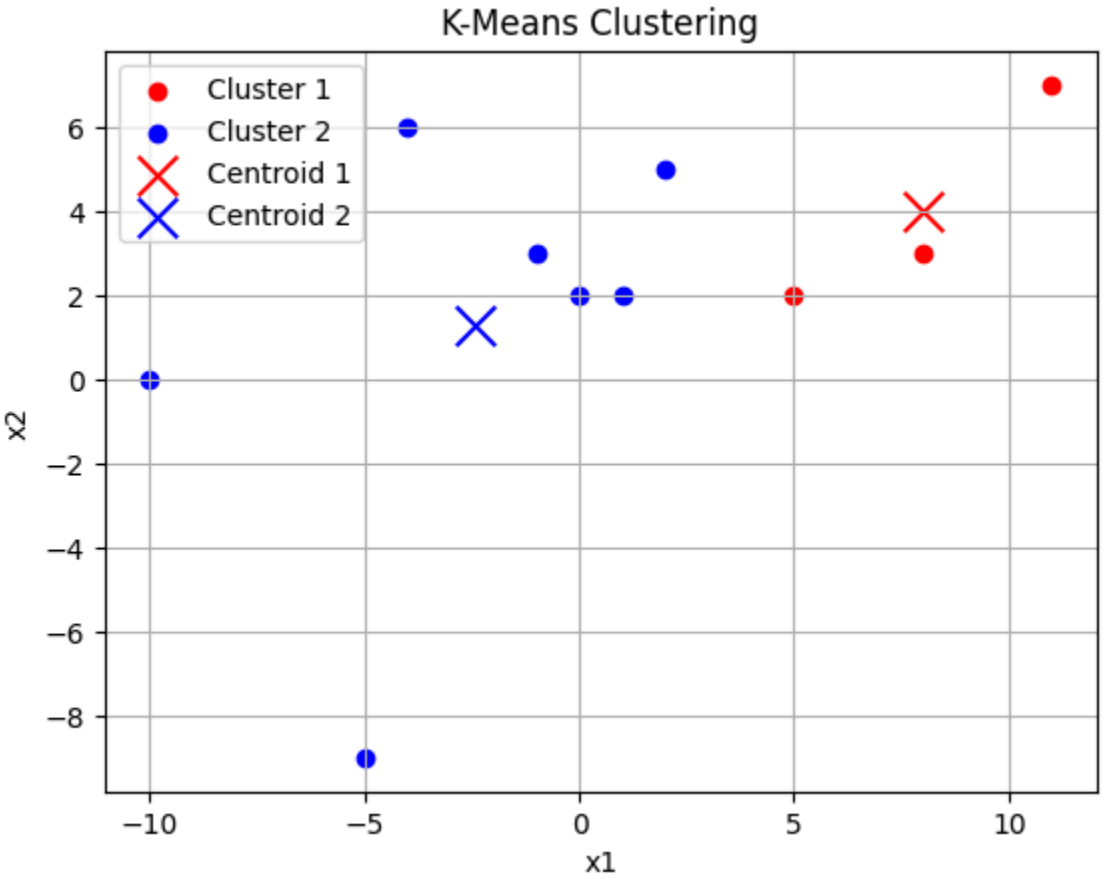
```
[11, 7] [0, 2]
```

In [17]:
```
itr = 0
while True:
    c1 = []
    c2 = []

    for j in range(len(x1)):
        p = [x1[j], x2[j]]
        d1 = eg(p, c1_cent)
        d2 = eg(p, c2_cent)
        if d1 <= d2:
            c1.append(p)
        else:
            c2.append(p)

    itr += 1
    print("C1:", c1)
    print("C2:", c2)

    c1_new = np.mean(c1, axis=0).tolist()
    c2_new = np.mean(c2, axis=0).tolist()
    print(c1_new, c2_new)

    if c1_new == c1_cent and c2_new == c2_cent:
        break
    else:
        c1_cent = c1_new
        c2_cent = c2_new
```

```
C1: [[11, 7], [8, 3]]
C2: [[1, 2], [2, 5], [5, 2], [-1, 3], [0, 2], [-4, 6], [-5, -9], [-10, 0]]
[9.5, 5.0] [-1.5, 1.375]
C1: [[11, 7], [8, 3], [5, 2]]
C2: [[1, 2], [2, 5], [-1, 3], [0, 2], [-4, 6], [-5, -9], [-10, 0]]
[8.0, 4.0] [-2.4285714285714284, 1.2857142857142858]
C1: [[11, 7], [8, 3], [5, 2]]
C2: [[1, 2], [2, 5], [-1, 3], [0, 2], [-4, 6], [-5, -9], [-10, 0]]
[8.0, 4.0] [-2.4285714285714284, 1.2857142857142858]
```

In [18]:
```
import matplotlib.pyplot as plt

c1 = np.array(c1)
c2 = np.array(c2)

plt.scatter(c1[:, 0], c1[:, 1], color='red', label='Cluster 1')
plt.scatter(c2[:, 0], c2[:, 1], color='blue', label='Cluster 2')
```

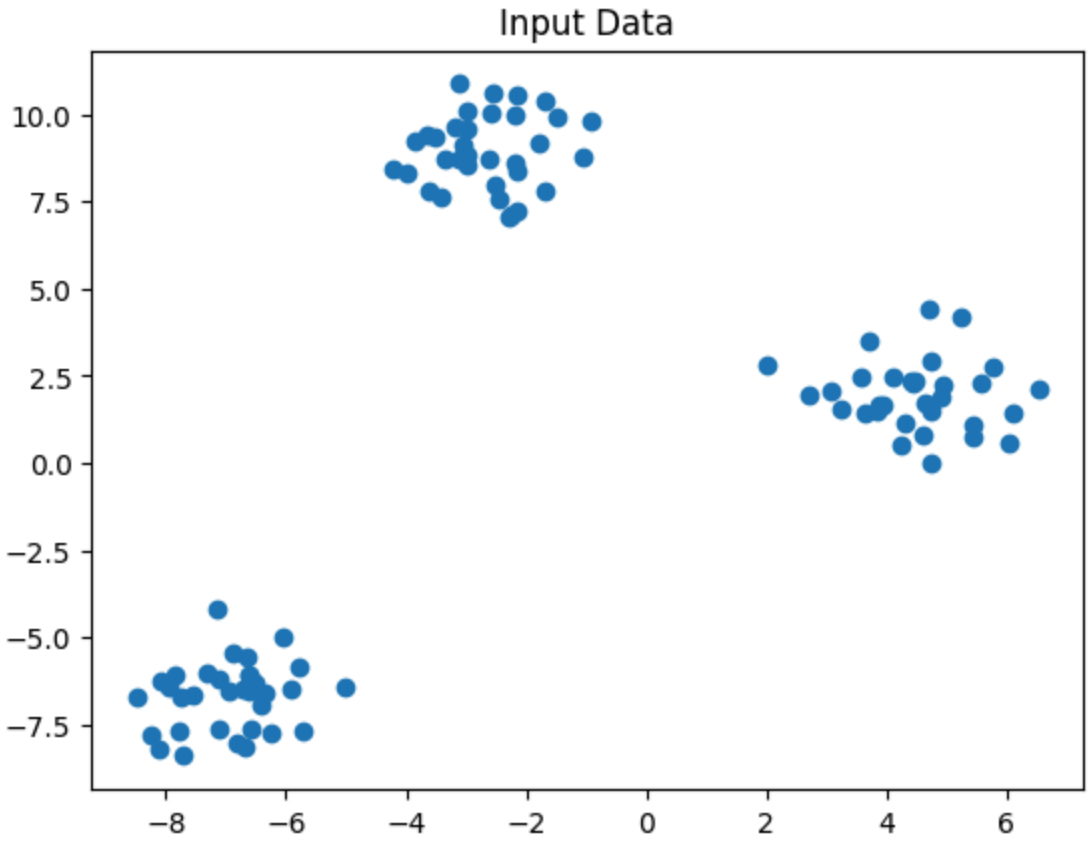
```
plt.scatter(c1_cent[0], c1_cent[1], color='red', marker='x', s=200, label='Cer
plt.scatter(c2_cent[0], c2_cent[1], color='blue', marker='x', s=200, label='Ce
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('K-Means Clustering')
plt.legend()
plt.grid(True)
plt.show()
```



Sklearn Metrics

In [1]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import pairwise_distances_argmin_min
import numpy as np
from sklearn.datasets import make_blobs
import pandas as pd
```

In [2]:
```
# Create sample 2D data
data, _ = make_blobs(n_samples=100, n_features=2, random_state=42)
plt.scatter(data[:, 0], data[:, 1])
plt.title("Input Data")
plt.show()
```



In [3]:
```python
def update_clusters(data, centroids, df):
    labels = pairwise_distances_argmin_min(data, centroids)[0]
    df["label"] = labels
    updated_centroids = df.groupby("label").mean().values
    return updated_centroids, labels


def kmeans(data, k=3, epoch=100):
    _, c = data.shape[:2]
    df = pd.DataFrame(data, columns=[i for i in range(c)])
    centroids = np.random.permutation(data)[:k]

    for _ in range(epoch):
        new_centroids, labels = update_clusters(data, centroids, df)
        if np.allclose(new_centroids, centroids):
            break
        centroids = new_centroids

    return centroids, labels
```

In [4]:
```python
centroids, labels = kmeans(data)
```

In [5]:
```python
plt.scatter(data[:, 0], data[:, 1], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 1], marker="*", s=120, c="black")
plt.title("K-Means Clustering Result")
plt.show()
```

Edit Distance

In [ ]:

```python
import numpy as np
import pandas as pd

# Input strings
str1 = "sunday"
str2 = "monday"

# Length +1 for extra row and column
s_l = len(str1) + 1
t_l = len(str2) + 1

# Create matrix with zeros
mat = np.zeros((s_l, t_l), dtype=np.uint8)
# print(mat)
# Fill first row
mat[0] = [i for i in range(t_l)]

# Fill first column
mat[:, 0] = [i for i in range(s_l)]
# print(mat)
# Function to calculate edit distance
def edit(str1, str2, mat, i, j):
```

```
    if str1[i-1] == str2[j-1]:
        return mat[i-1][j-1]

    else:
        return min(
            mat[i-1][j-1],   # Replace
            mat[i-1][j],     # Delete
            mat[i][j-1]      # Insert
        ) + 1

# Fill matrix
for i in range(1, s_l):
    for j in range(1, t_l):
        mat[i, j] = edit(str1, str2, mat, i, j)

# Convert into table
df = pd.DataFrame(
    mat,
    index=list("_" + str1),
    columns=list("_" + str2)
)

print(df)

# Final edit distance
print("\nEdit Distance =", mat[s_l-1][t_l-1])
```

```
[[0 1 2 3 4 5 6]
 [1 0 0 0 0 0 0]
 [2 0 0 0 0 0 0]
 [3 0 0 0 0 0 0]
 [4 0 0 0 0 0 0]
 [5 0 0 0 0 0 0]
 [6 0 0 0 0 0 0]]
   _  m  o  n  d  a  y
_  0  1  2  3  4  5  6
s  1  1  2  3  4  5  6
u  2  2  2  3  4  5  6
n  3  3  3  2  3  4  5
d  4  4  4  3  2  3  4
a  5  5  5  4  3  2  3
y  6  6  6  5  4  3  2

Edit Distance = 2
```

Fuzzy

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import math as mp
import seaborn as sns
```

```
from sklearn.metrics import pairwise_distances_argmin,accuracy_score,silhouett
```

In [24]:

```
class fuzzy:

    def __init__(self,k=2):
        self.k = k


    def initial_wt(self,k,x_train):
        weight=np.random.RandomState(2).dirichlet(np.ones(k),size=len(self.x_t
        weight_arr=np.array(weight)

        return weight_arr

    def labels_allot(self,x_train,centroid):
        self.labels = pairwise_distances_argmin(x_train,centroid)
        return self.labels

    def centroids(self,x_train,weight,m=2):
        centroid=[]
        self.centroid=centroid
        for i in range(self.k):
            cent=[]
            for j in range(len(x_train[0])):
                num = np.sum(x_train[:,j]*(weight[:,i]**m))
                deno = np.sum(weight[:,i]**m)
                cordinate = num/deno
                cent.append(np.trunc(cordinate*10) / 10)
            centroid.append(cent)

        return centroid

    def distance(self,x_train,centroid):
        distance=[]
        for i in range(len(x_train)):
            dis=[]
            for j in centroid:
                euc=mp.dist(x_train[i],j)
                dis.append(np.trunc(euc*100)/100)
            distance.append(dis)

        return distance

    def new_wt(self,distance,m=2):
        weight=[]
        for i in distance:
            wt=[]
            for j in range(len(distance[0])):
                i=np.array(i)
                num=1/(i[j])
                deno=np.sum(1/i)
                wt.append(np.round((num/deno),2))
            weight.append(wt)
```

```
        return np.array(weight)

    def fit(self,x_train):
        x_train=x_train.to_numpy()
        self.x_train = x_train
        visual_cent = []
        visual_label = []


        initial_wt=self.initial_wt(self.k,x_train)
        initial_cent=self.centroids(x_train,initial_wt)
        cent=initial_cent

        visual_cent.append(cent)
        labels = self.labels_allot(x_train,cent)
        visual_label.append(labels)

        count=0
        while (count<100):
            new_w=self.new_wt(self.distance(x_train,cent))
            new_cent=self.centroids(x_train,new_w)
            visual_cent.append(new_cent)
            labels = self.labels_allot(x_train,new_cent)
            visual_label.append(labels)
            if cent==new_cent:
                break
            else:
                cent=new_cent
                count+=1

        self.visual = visual_cent
        self.visual_lab = visual_label


    def visualization(self):
        # for k=2 only and only for two features
        centroid = self.visual
        x_train = self.x_train
        labels=np.array(self.visual_lab)
        centroid=np.array(centroid)

        print("initial centroids : ")
        print(centroid[0])
        plt.title("Initial Centoids")
        sns.scatterplot(x=x_train[:,0],y=x_train[:,1],hue=labels[0])
        sns.scatterplot(x=centroid[0][:,0], y=centroid[0][:,1],marker="*",hue=
        plt.xlabel('Feature 1')
        plt.ylabel('Feature 2')
        plt.legend()
        plt.show()

        for i in range(1,len(centroid)):
```

```
                print("updated centroids : ")
                print(centroid[i])
                plt.title("Centoids after {} iteration".format(i))

                sns.scatterplot(x=x_train[:,0],y=x_train[:,1],hue=labels[i])
                sns.scatterplot(x=centroid[i][:,0], y=centroid[i][:,1],marker=
                plt.xlabel('Feature 1')
                plt.ylabel('Feature 2')
                plt.legend()
                plt.show()
```

In [ ]:

In [14]:
```
from sklearn import datasets
```

In [15]:
```
x,y=datasets.make_blobs(n_samples=500, centers=None, n_features=2,random_state
```

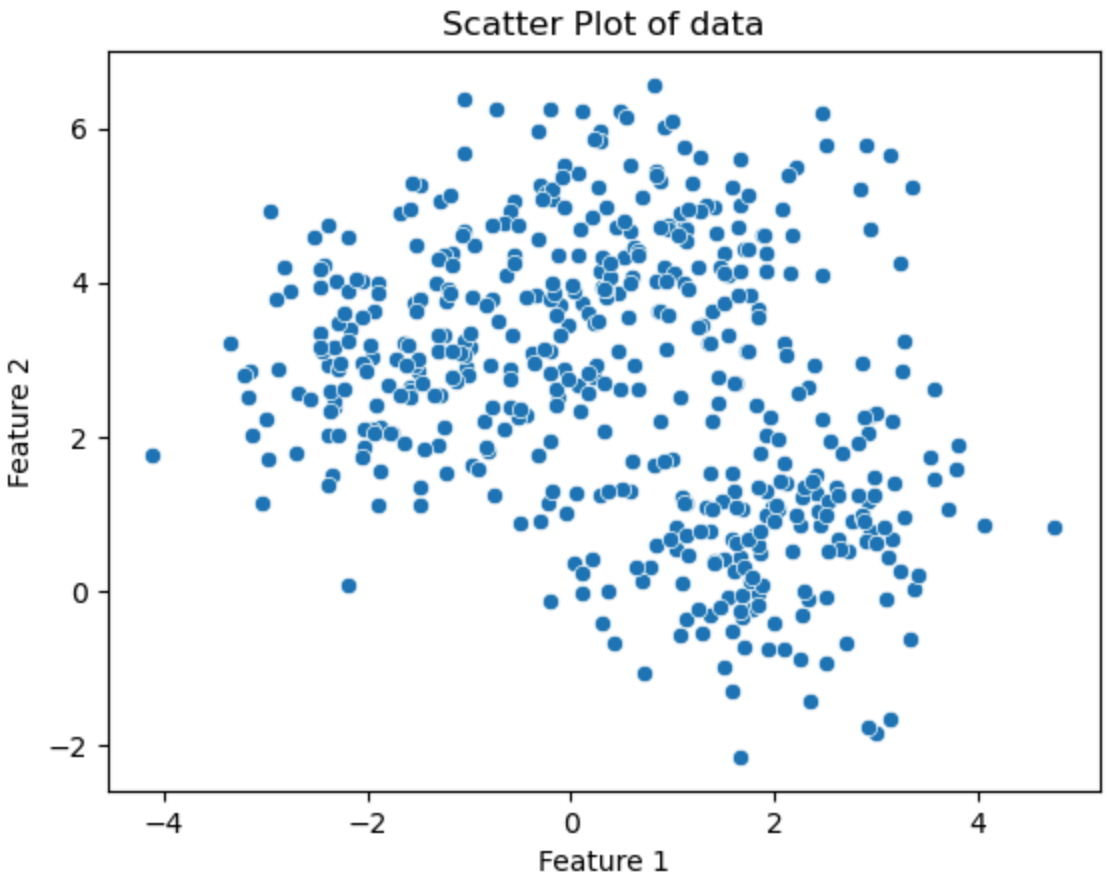
In [16]:
```
df=pd.DataFrame(x,columns=["Feature1","Feature2"])
data=df.copy()
data["Target"]=y
data[["Feature1","Feature2"]]
```

Out[16]:

	Feature1	Feature2
0	1.103182	4.705777
1	-1.932846	3.642251
2	-2.034422	1.866002
3	1.616402	2.686831
4	-0.960010	4.492566
...	...	...
495	2.660388	1.793220
496	2.875589	2.257612
497	2.826673	1.927102
498	-0.094483	5.358239
499	2.507757	0.995560

500 rows × 2 columns

In [17]:
```
plt.title("Scatter Plot of data")
sns.scatterplot(x=x[:,0],y=x[:,1])
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



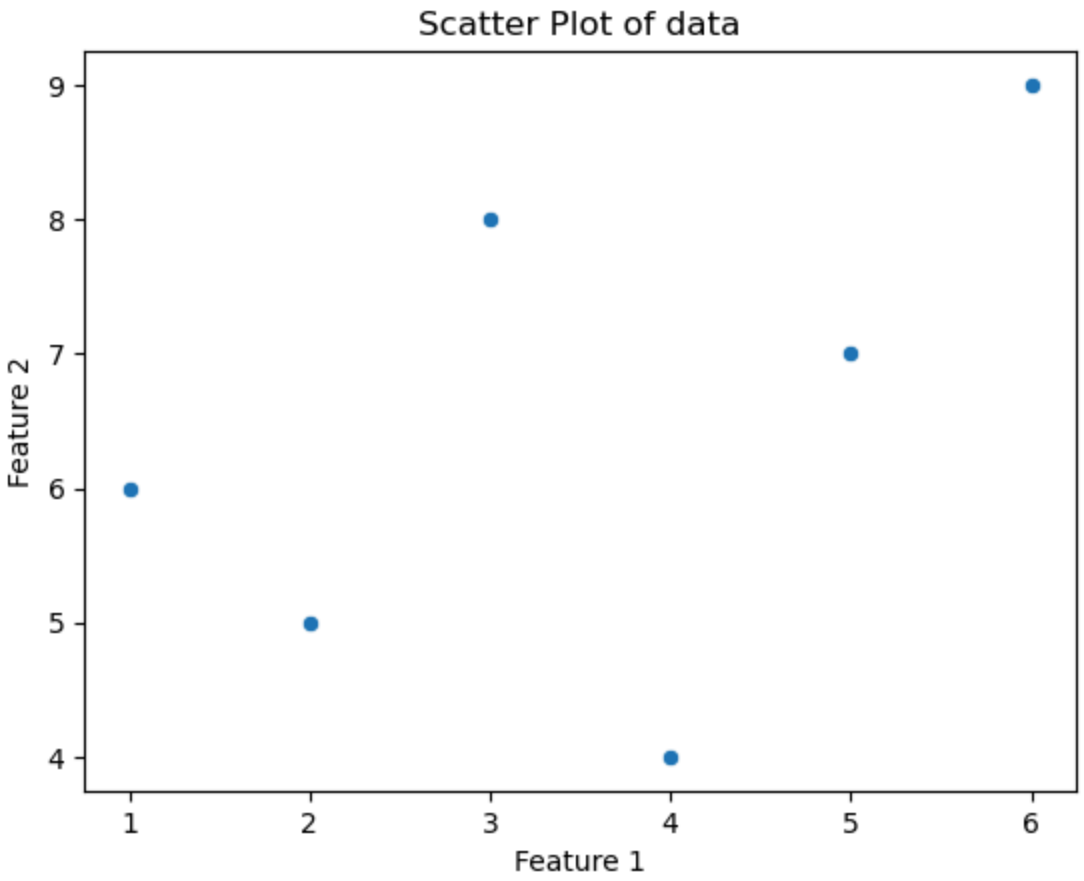
In [18]:
```
df=pd.DataFrame()
```

In [19]:
```
df["X"]=[1,2,3,4,5,6]
df["Y"]=[6,5,8,4,7,9]
df
```

Out[19]:

	X	Y
0	1	6
1	2	5
2	3	8
3	4	4
4	5	7
5	6	9

In [20]:
```
plt.title("Scatter Plot of data")
sns.scatterplot(x=df["X"].values,y=df["Y"].values)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

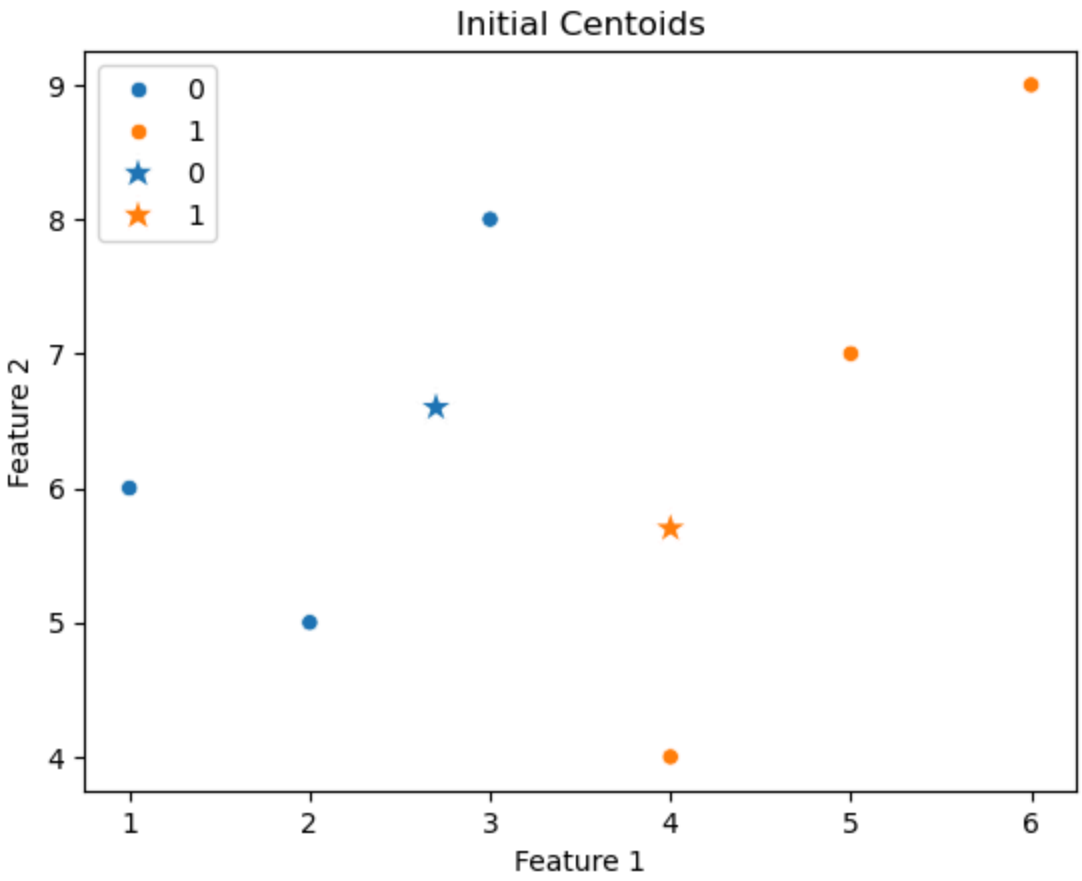


In [21]:
```
fz=fuzzy(k=2)
```

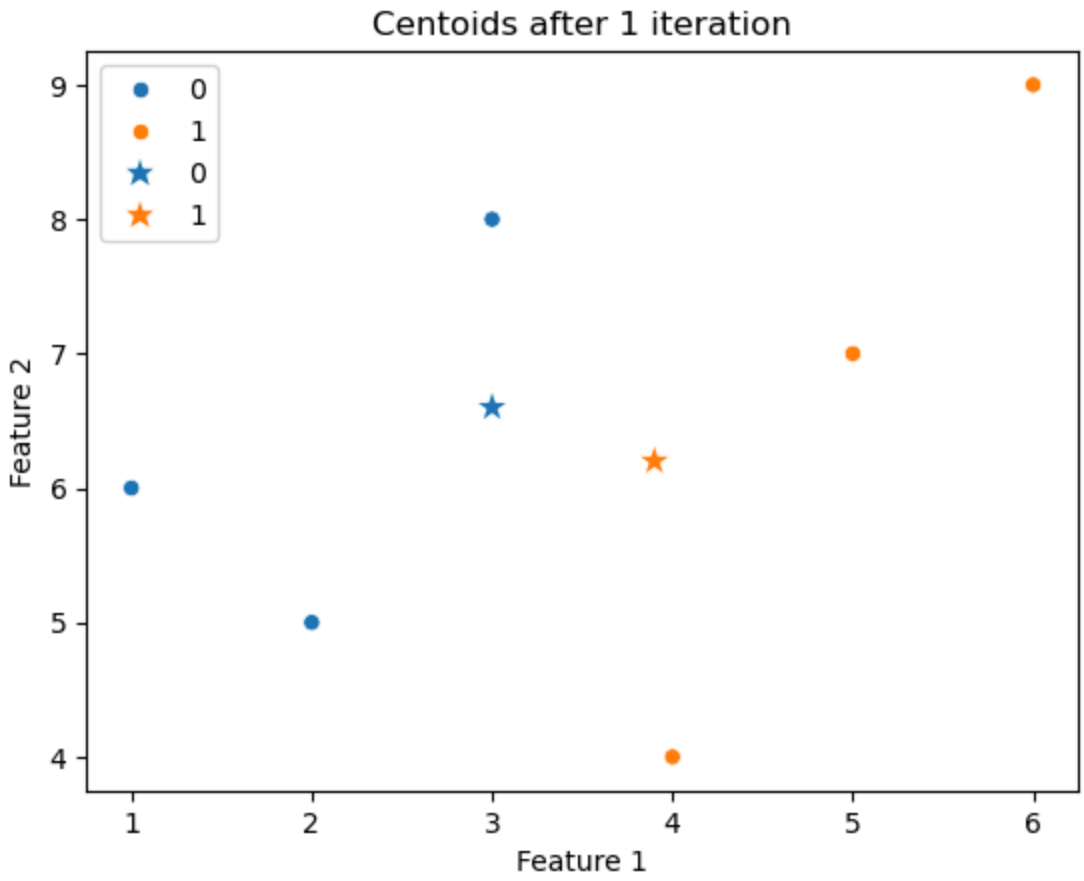
In [22]:
```
fz.fit(df[["X","Y"]])
```

In [23]:
```
fz.visualization()
```

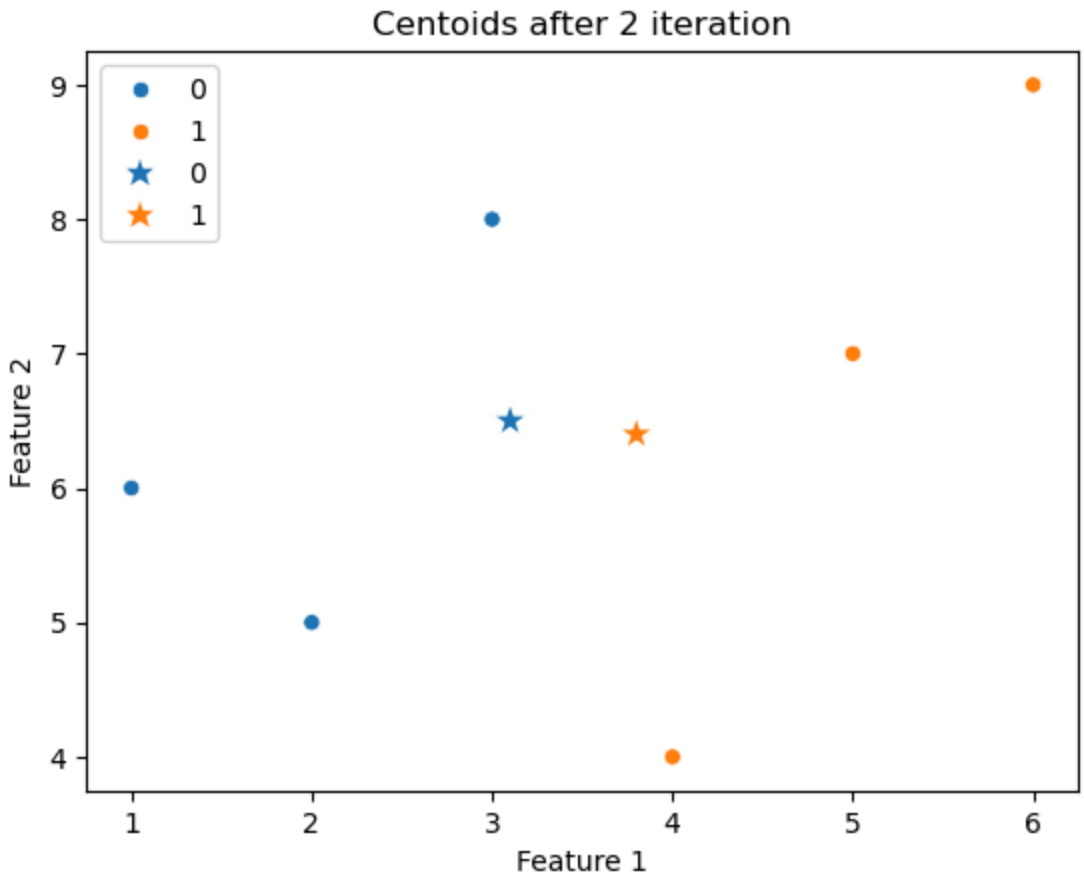
```
initial centroids :
[[2.7 6.6]
 [4.  5.7]]
```



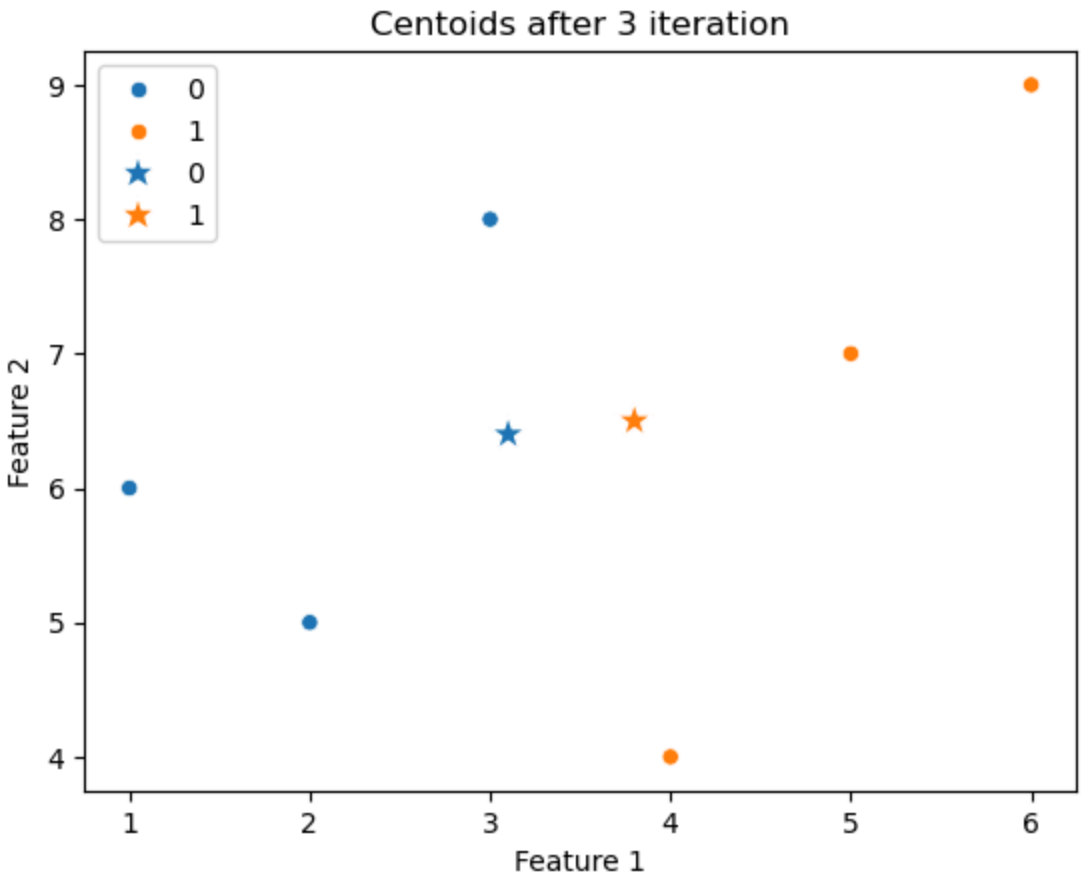

```
updated centroids :
[[3.  6.6]
 [3.9 6.2]]
```



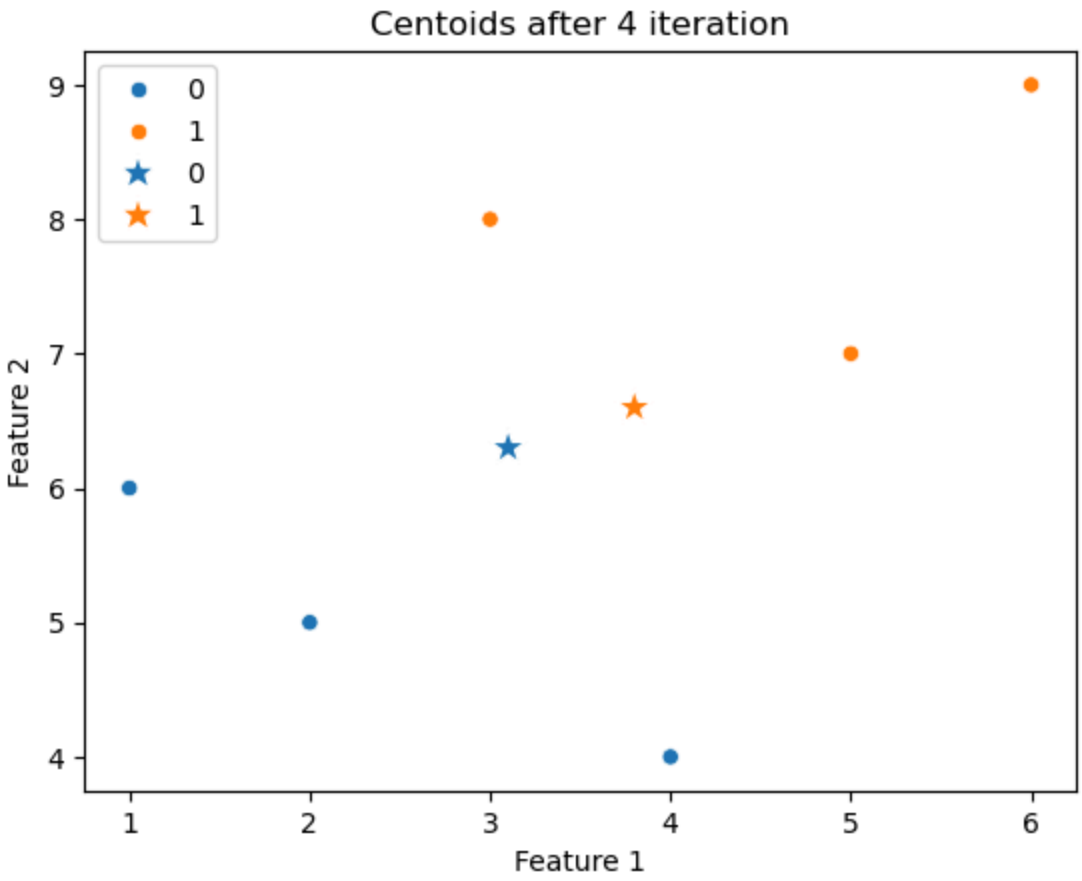

```
updated centroids :
[[3.1 6.5]
 [3.8 6.4]]
```



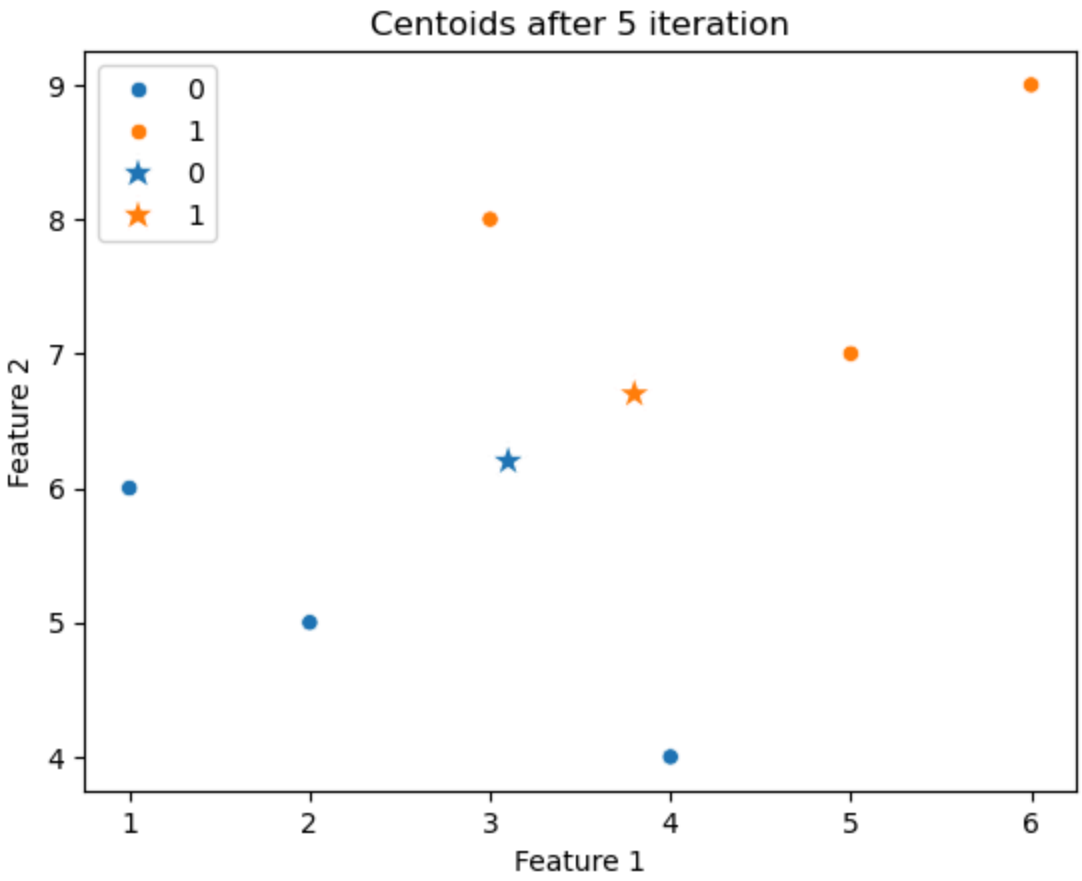

```
updated centroids :
[[3.1 6.4]
 [3.8 6.5]]
```



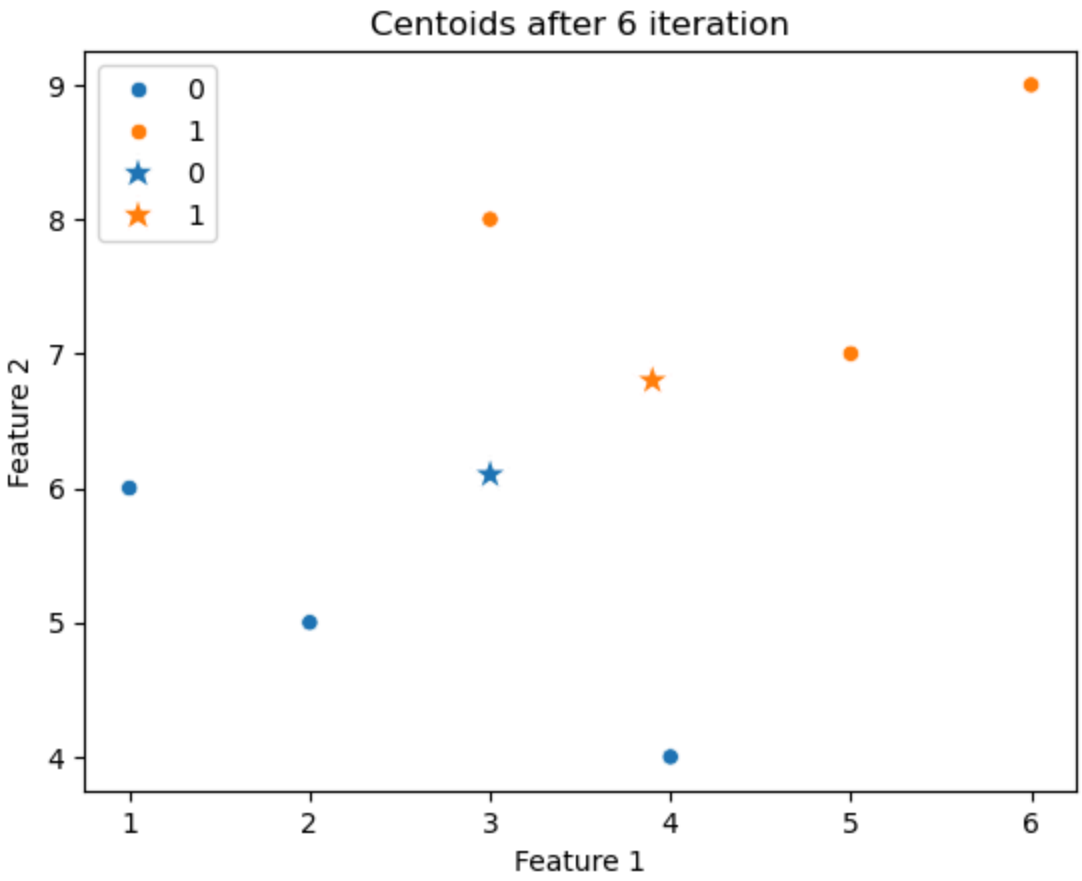

```
updated centroids :
[[3.1 6.3]
 [3.8 6.6]]
```



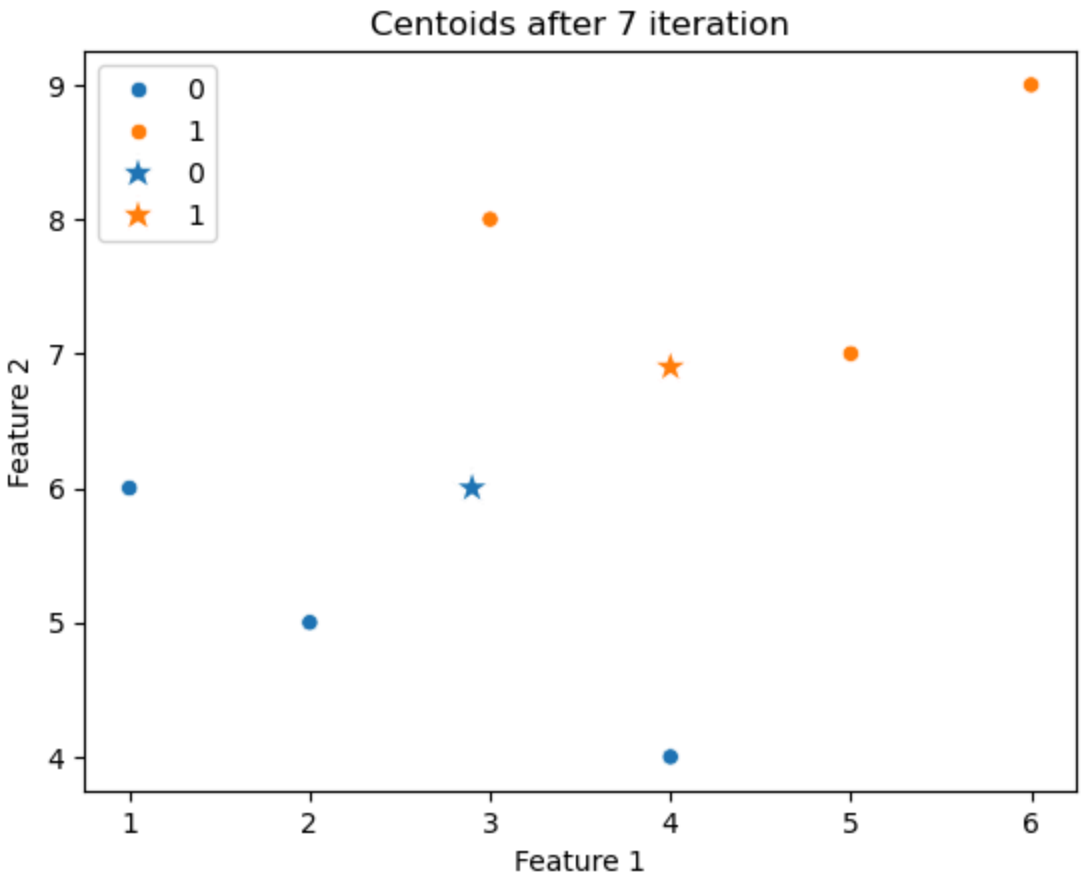

```
updated centroids :
[[3.1 6.2]
 [3.8 6.7]]
```



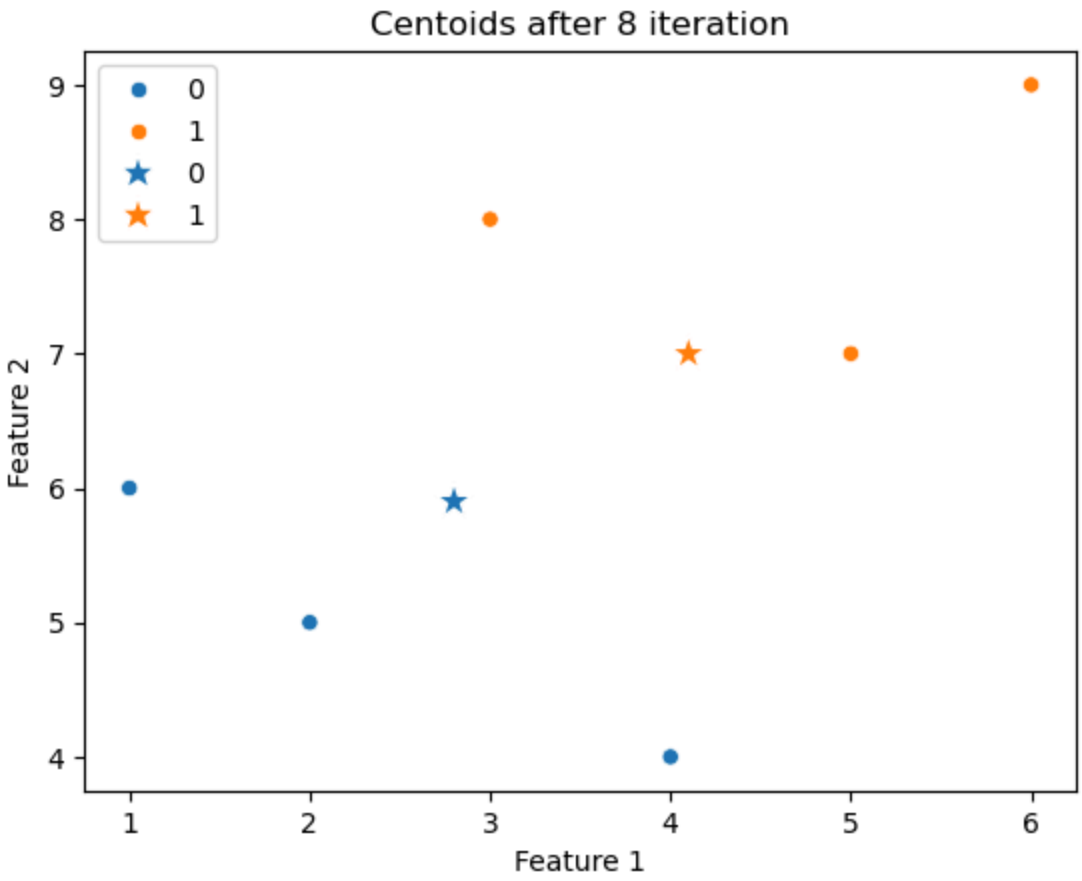

```
updated centroids :
[[3.  6.1]
 [3.9 6.8]]
```



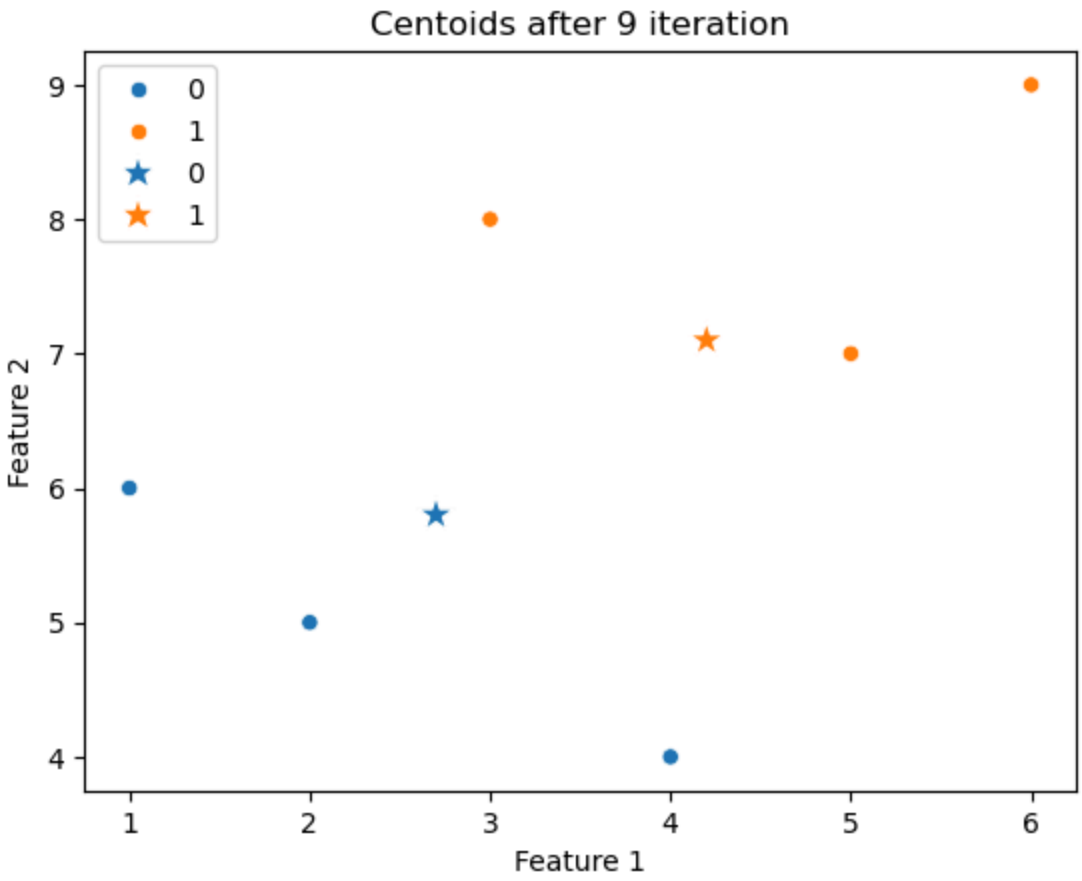

```
updated centroids :
[[2.9 6. ]
 [4.  6.9]]
```



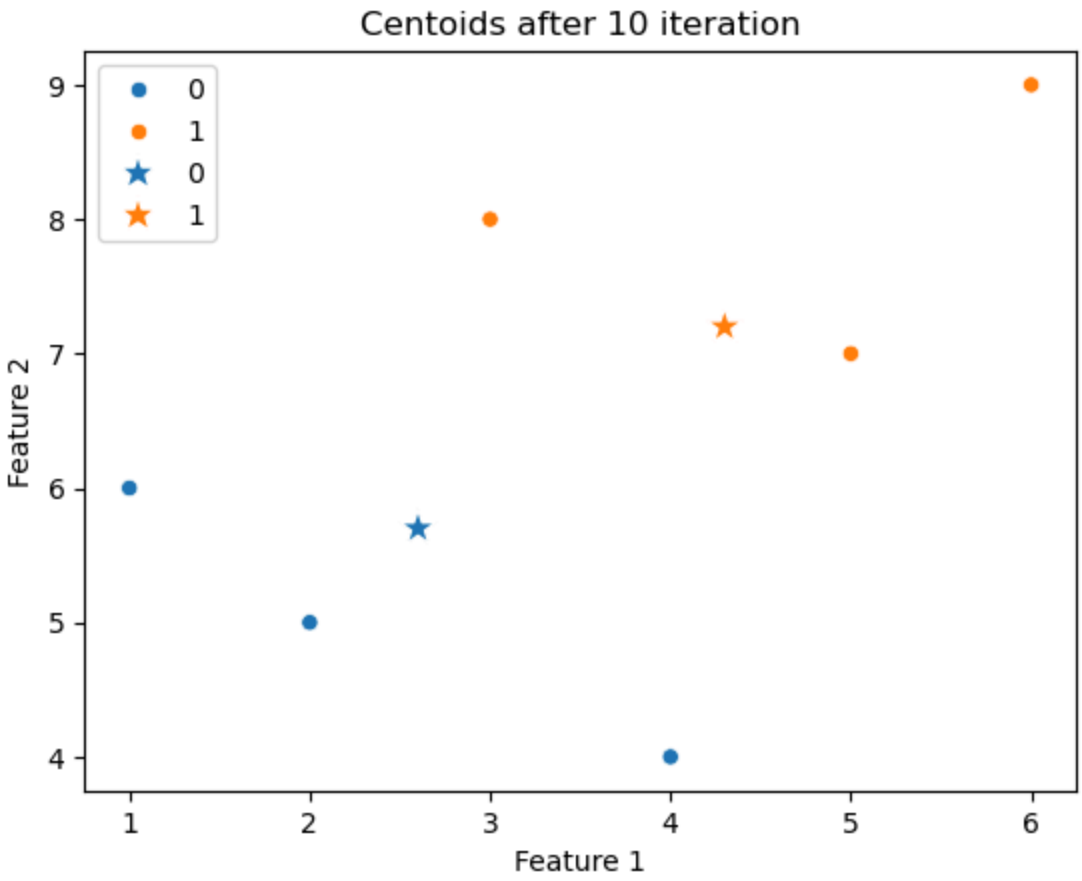

```
updated centroids :
[[2.8 5.9]
 [4.1 7. ]]
```



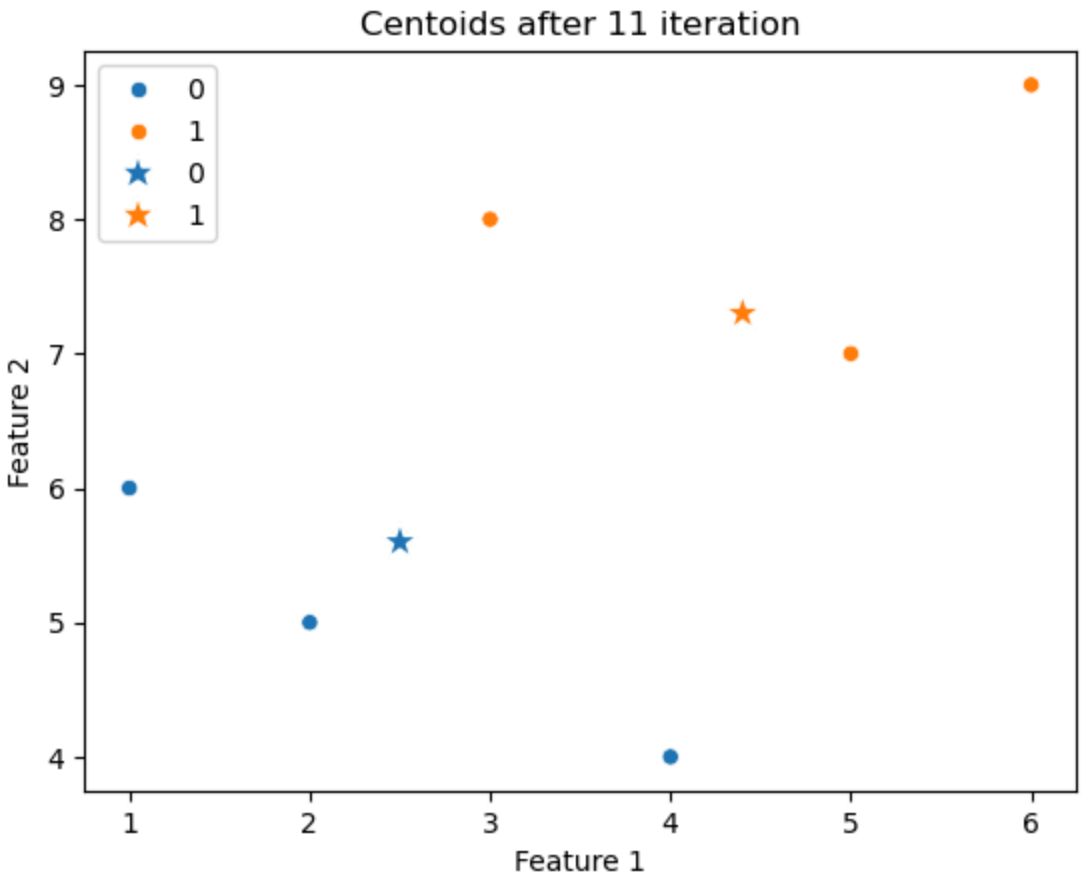

```
updated centroids :
[[2.7 5.8]
 [4.2 7.1]]
```



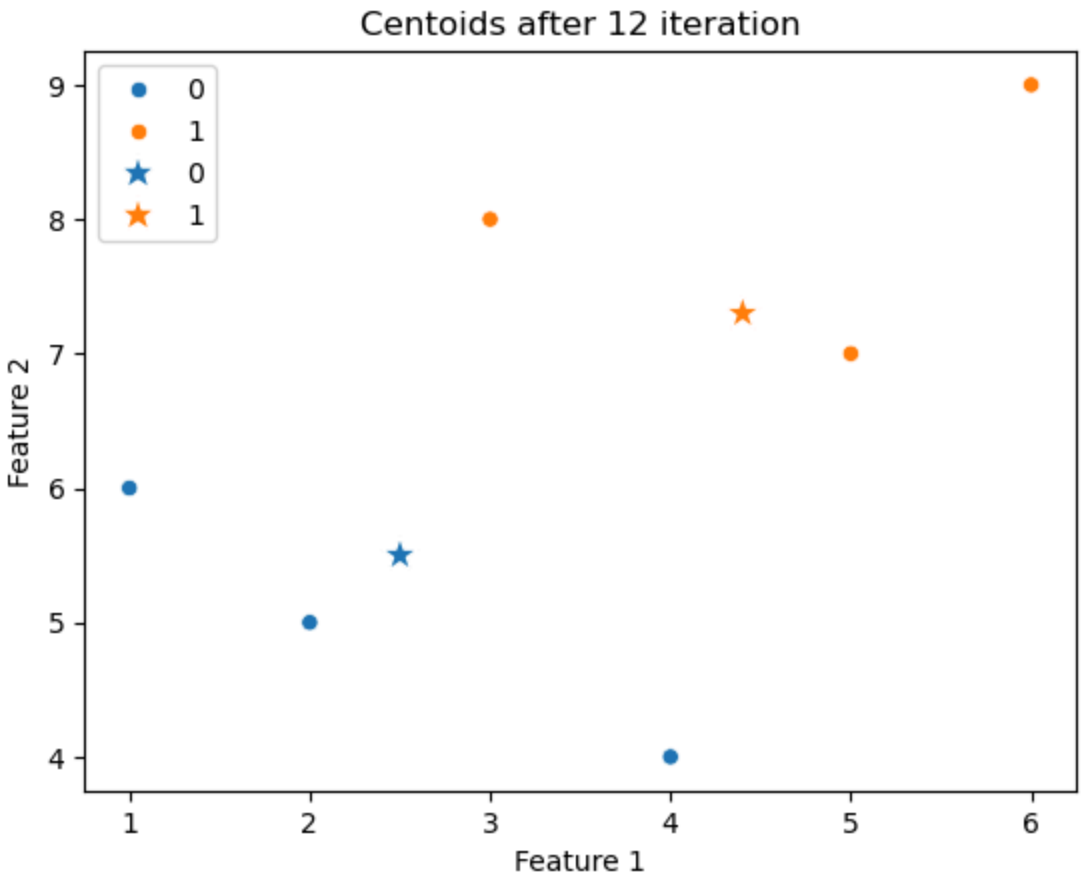

```
updated centroids :
[[2.6 5.7]
 [4.3 7.2]]
```



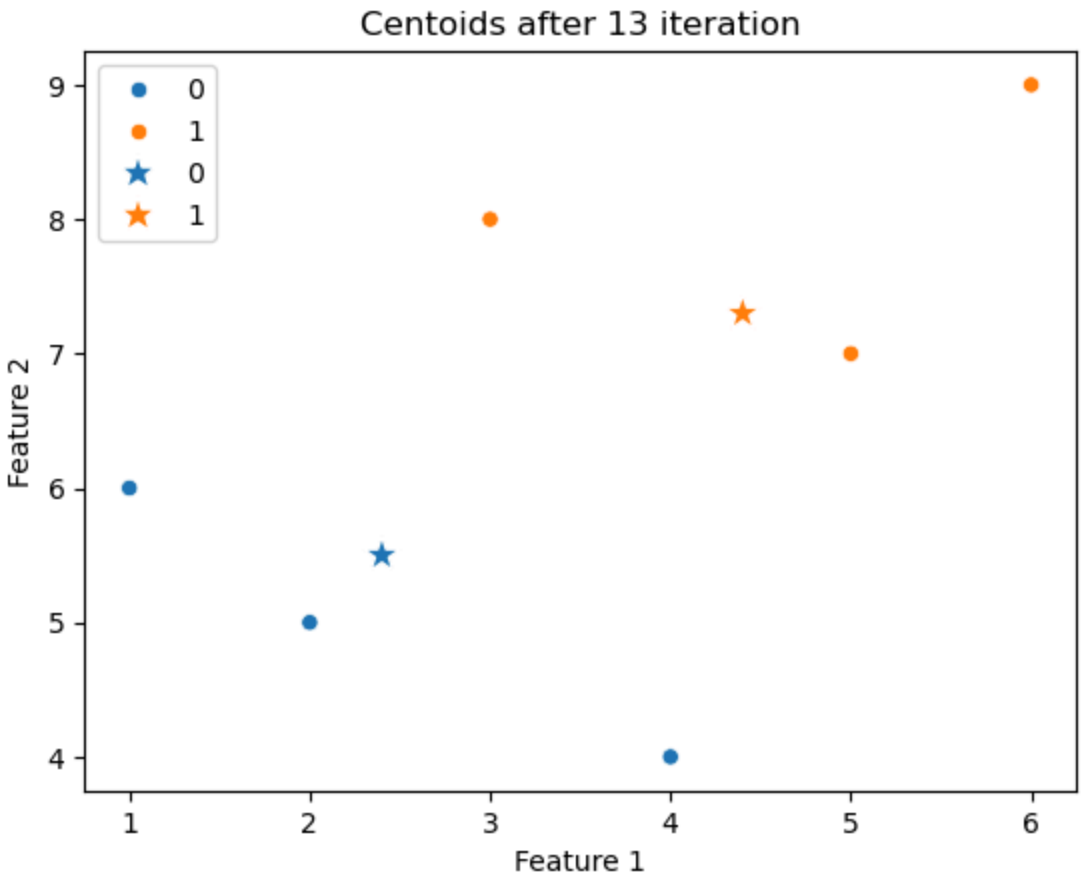

```
updated centroids :
[[2.5 5.6]
 [4.4 7.3]]
```



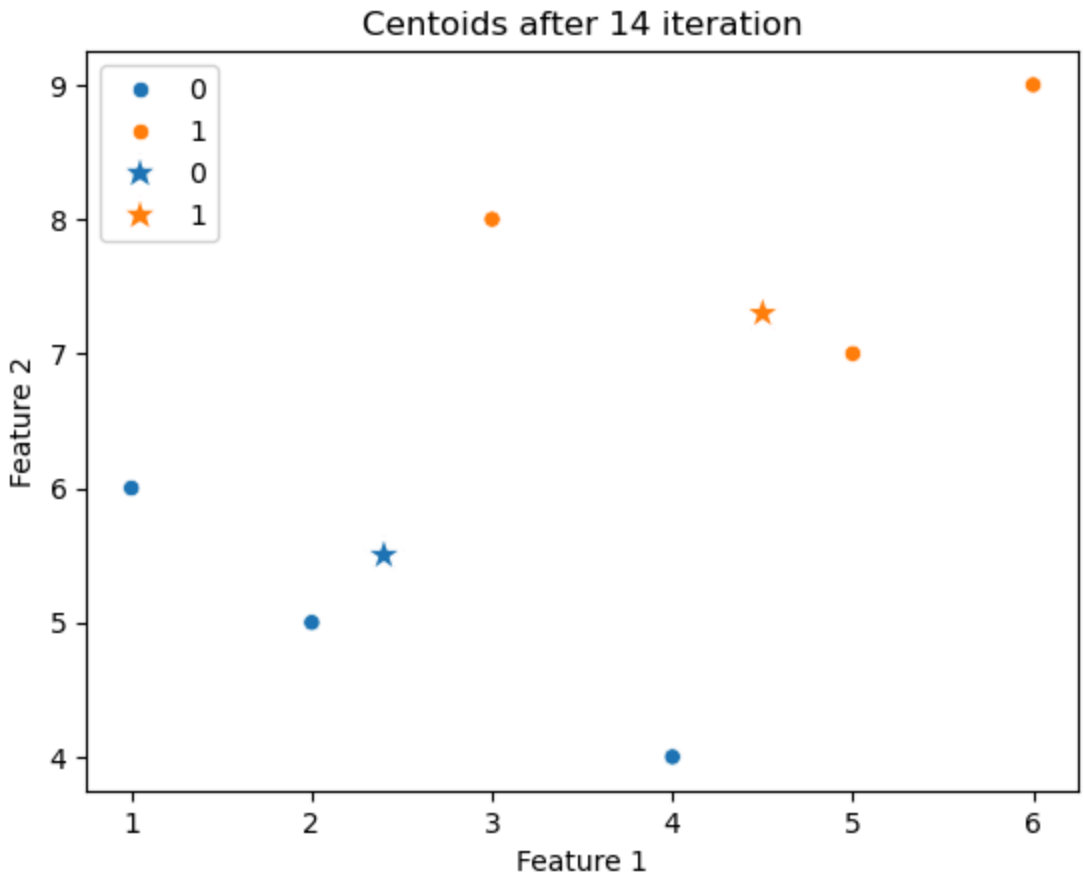

```
updated centroids :
[[2.5 5.5]
 [4.4 7.3]]
```



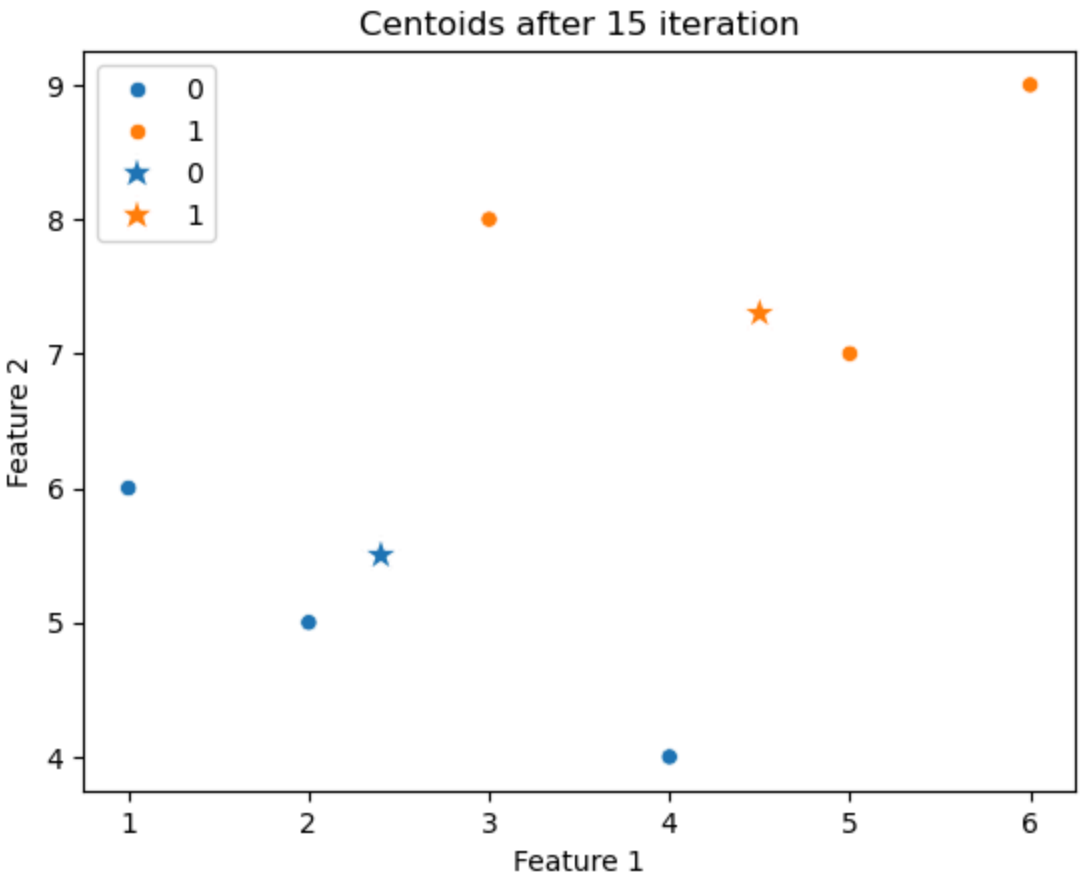

```
updated centroids :
[[2.4 5.5]
 [4.4 7.3]]
```




```
updated centroids :
[[2.4 5.5]
 [4.5 7.3]]
```




```
updated centroids :
[[2.4 5.5]
 [4.5 7.3]]
```


Centoids after 15 iteration
0
1
0
1
Feature 2
Feature 1
9
8
7
6
5
4
1
2
3
4
5
6

In [ ]:

In [ ]:

In [ ]: